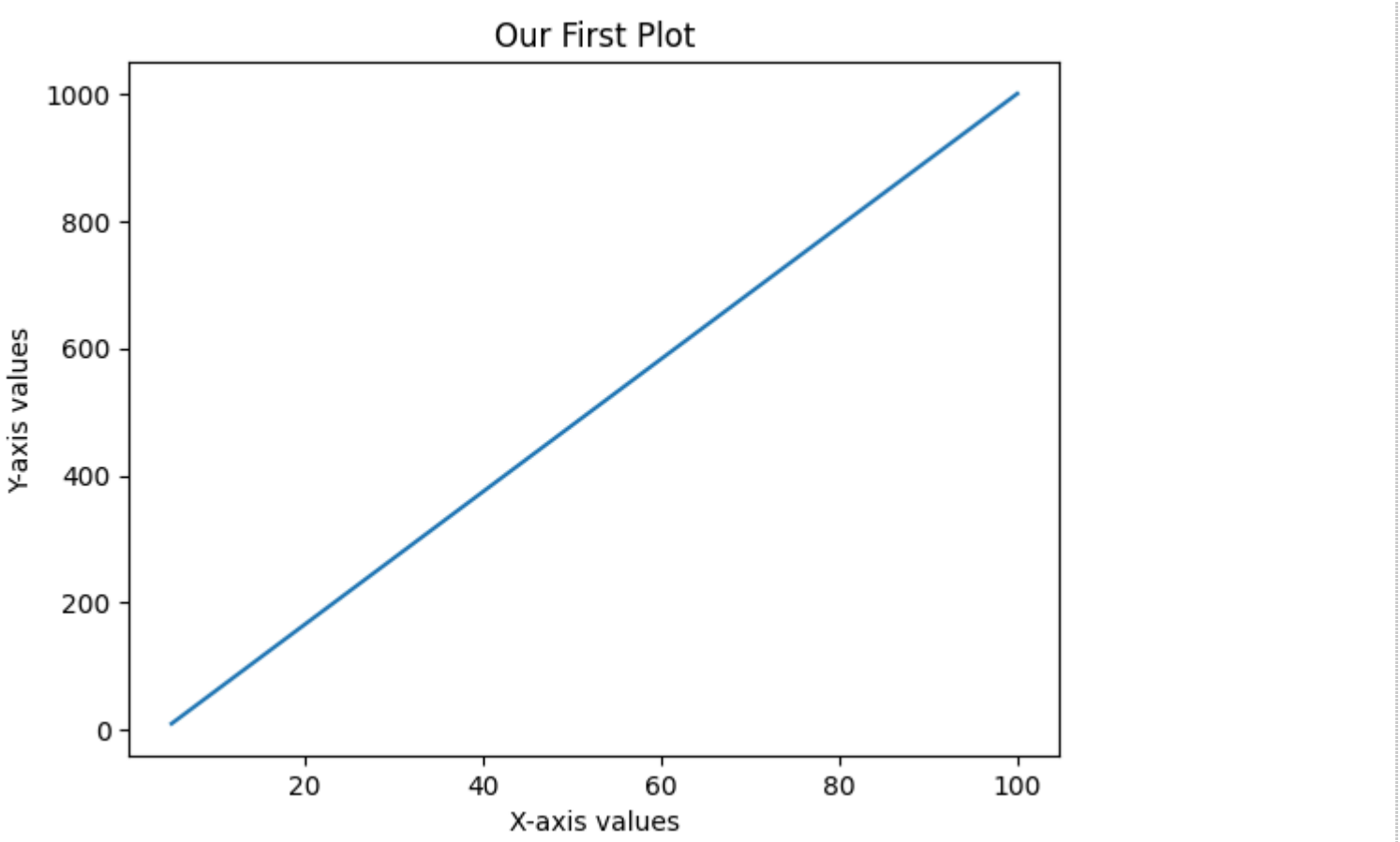
Plotting with `matplotlib`

`matplotlib` is a python library. It contains the `pyplot` module, which is basically a collection of functions such as `plot`, `title`, `show()` etc. `pyplot` is one of the most commonly used module for creating a variety of plots such as line plots, bar plots, histograms etc.

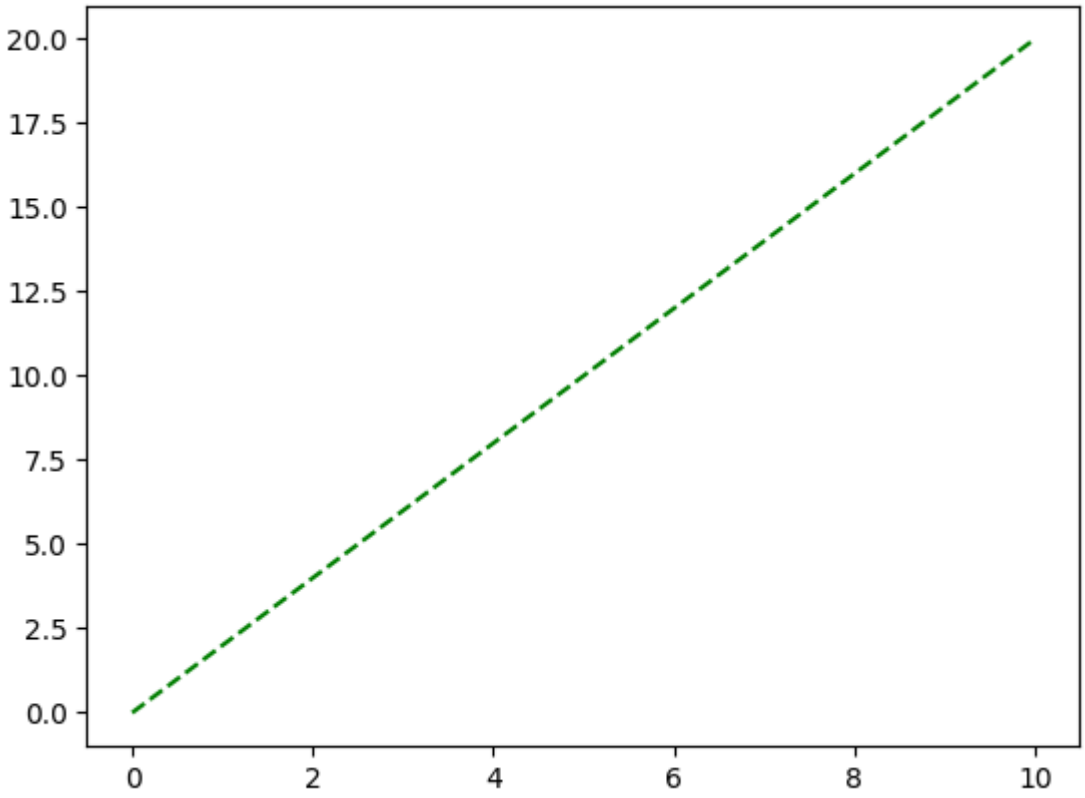
[]:
```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

[]:
```
# Plotting two 1-D numpy arrays
x = np.linspace(5, 100, 25)
y = np.linspace(10, 1000, 25)

plt.plot(x,y)
plt.xlabel("X-axis values")
plt.ylabel("Y-axis values")
plt.title("Our First Plot")
plt.show()
```

[]:


[]:
```
x = np.linspace(0, 10, 20)
y = x*2

plt.plot(x,y,'g--')
#plt.plot(x, y**2, 'r+')
plt.show()
```

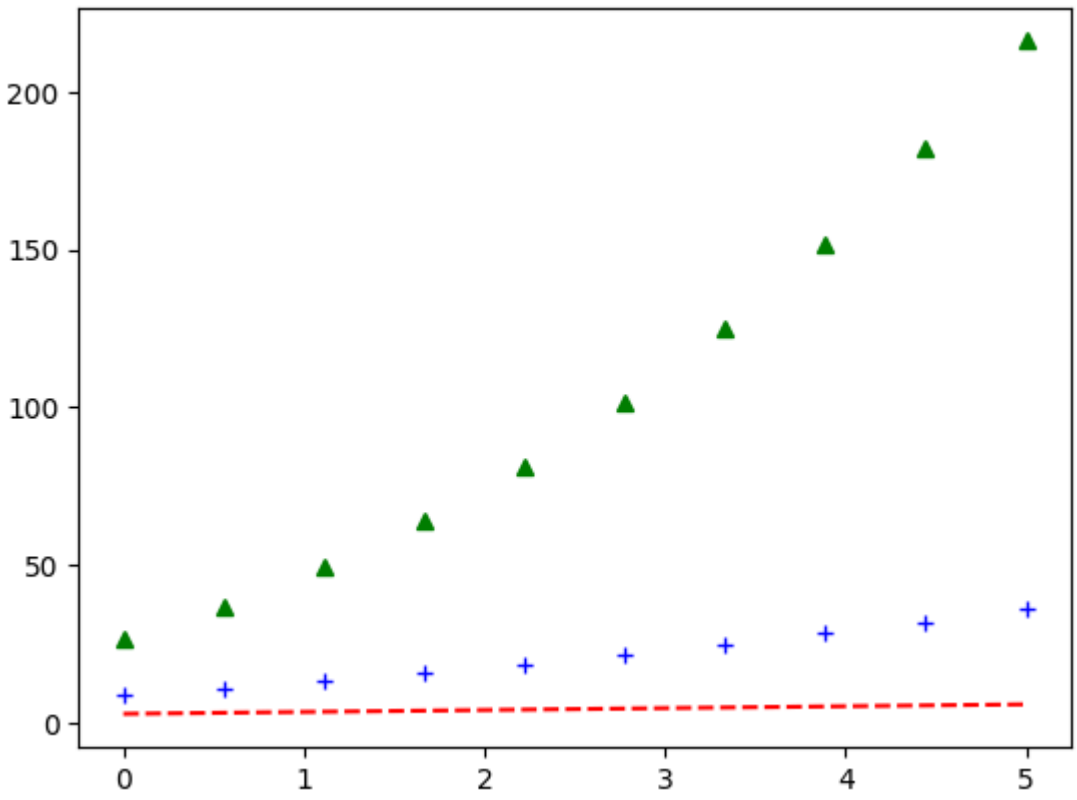
[]:



[]:

```
x = np.linspace(0, 5, 10)
y = np.linspace(3, 6, 10)

# plot three curves: y, y**2 and y**3 with different line types
plt.plot(x, y, 'r--')
plt.plot(x, y**2, 'b+')
plt.plot(x, y**3, 'g^')

# adding the legend in the plot
#plt.legend()
plt.show()
```

[]:



Figures and Subplots

You often need to create multiple plots in the same figure.

`matplotlib` has the concept of **figures and subplots** using which you can create *multiple subplots inside the same figure*.

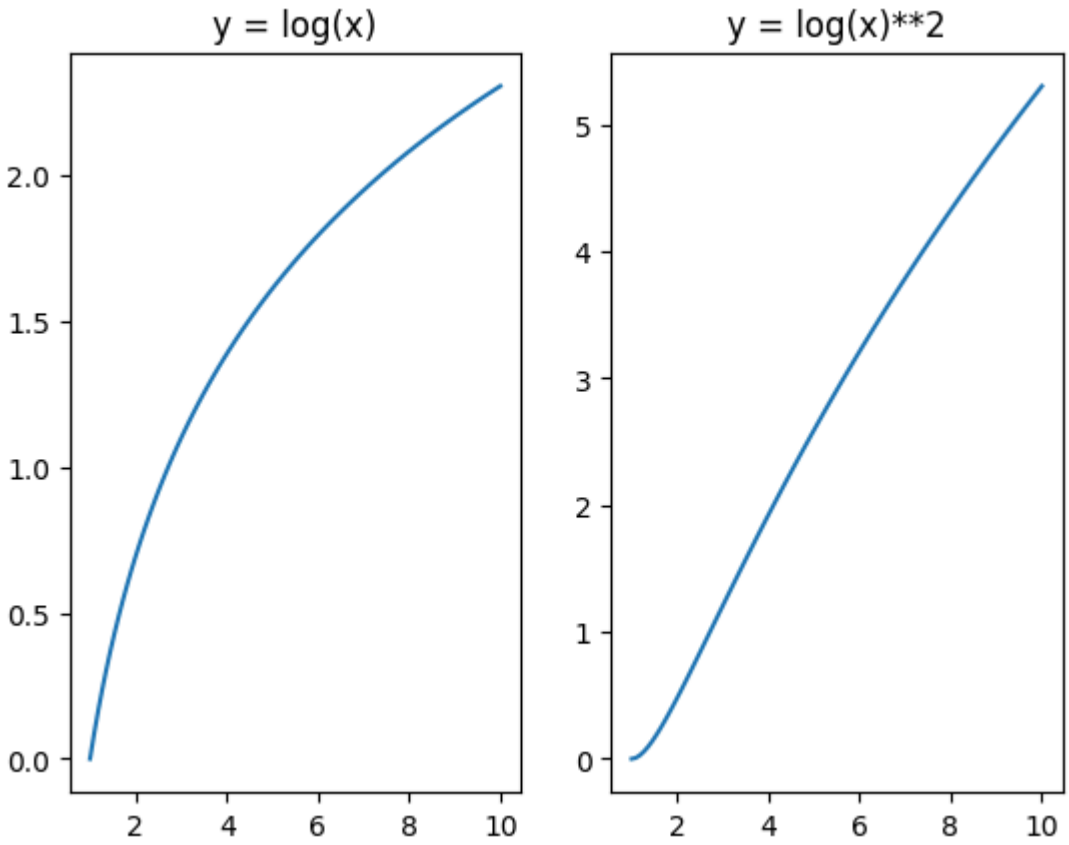
To create multiple plots in the same figure, you can use the method `plt.subplot(nrows, ncols, nsubplot)`.

[]:

```
x = np.linspace(1, 10, 100)
y = np.log(x)
```

```
# initiate a new figure explicitly
plt.figure(1)

# Create a subplot with 1 row, 2 columns

# create the first subplot in figure 1
# equivalent to plt.subplot(1, 2, 1)
plt.subplot(1,2,1)
plt.title("y = log(x)")
plt.plot(x, y)

# create the second subplot in figure 1
plt.subplot(1,2,2)
plt.title("y = log(x)**2")
plt.plot(x, y**2)
plt.show()
```

[]:

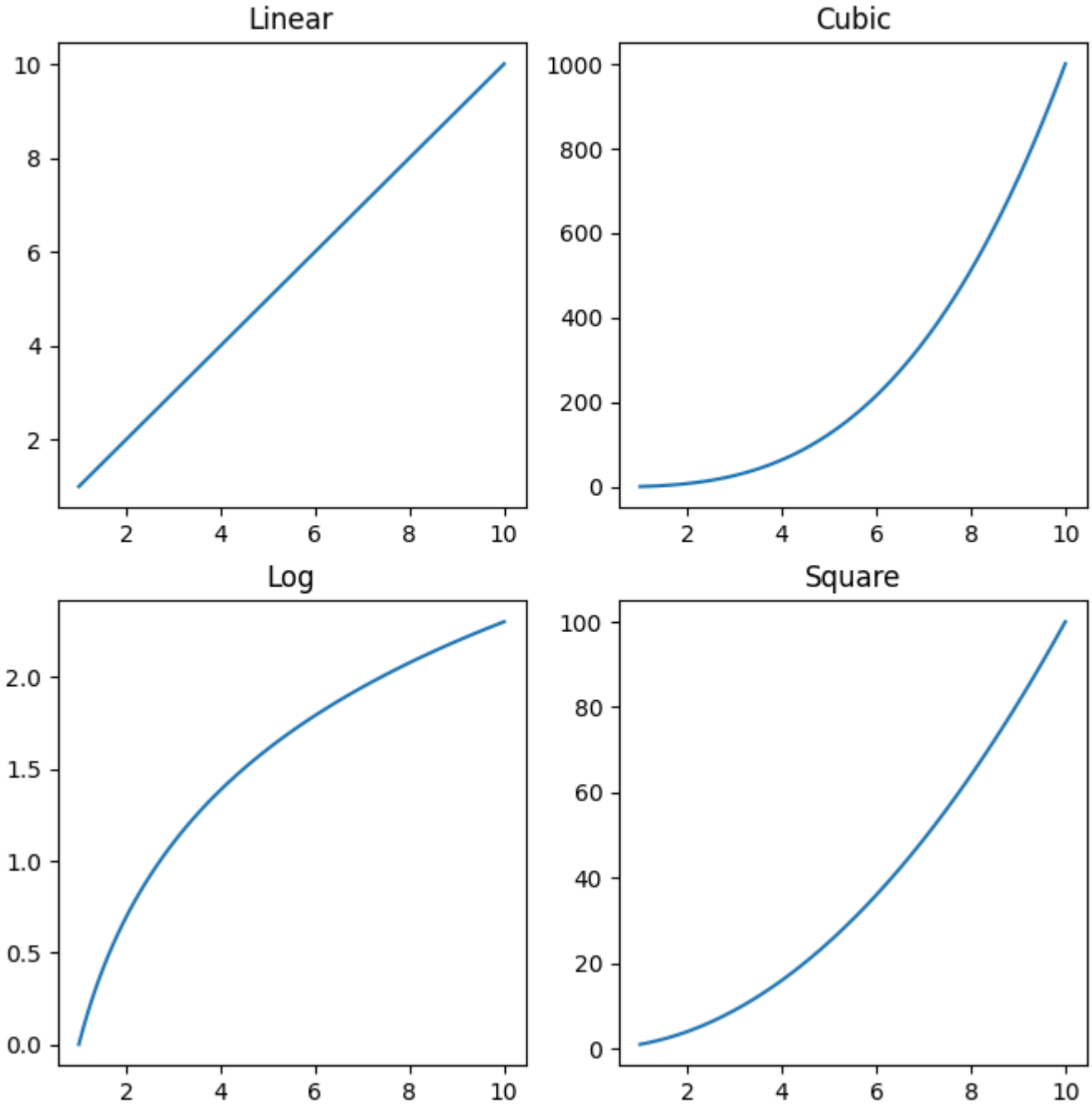


Let's see another example - say you want to create 4 subplots in two rows and two columns.

[]:

```
# Example: Create a figure having 4 subplots
x = np.linspace(1, 10, 100)

plt.figure(figsize=(8, 8))
# Optional command, since matplotlib creates a figure by default anyway
#plt.figure(1)

# subplot 1
plt.subplot(2,2,1)
plt.title("Linear")
plt.plot(x, x)

# subplot 2
plt.subplot(2,2,2)
plt.title("Cubic")
plt.plot(x, x**3)

#plt.figure(figsize=(3, 3))

# subplot 3
#plt.figure(2)

plt.subplot(2,2,3)
plt.title("Log")
plt.plot(x, np.log(x))

# subplot 4
plt.subplot(2,2,4)
plt.title("Square")
plt.plot(x, x**2)

plt.show()
```

[]:



You can see the list of colors and shapes here: https://matplotlib.org/api/pyplot_api.html#matplotlib.pyplot.plot

Types of Plots

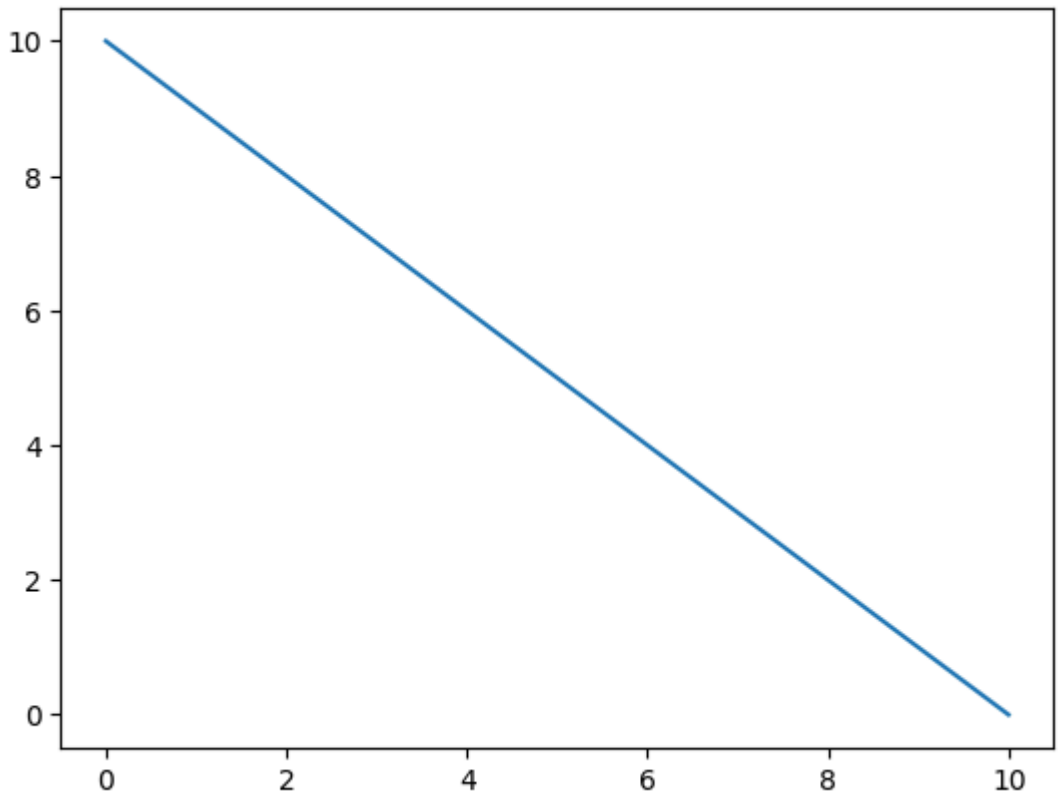
1. Line Plot

Line plot can be used when we are trying to get the trends over a period of time, eg. Unemployment in India over the last 10 years.

[]:

```
x = np.linspace(0, 10, 20)
y = np.linspace(10, 0, 20)

plt.plot(x,y)

# show the plot
plt.show()
```

[]:



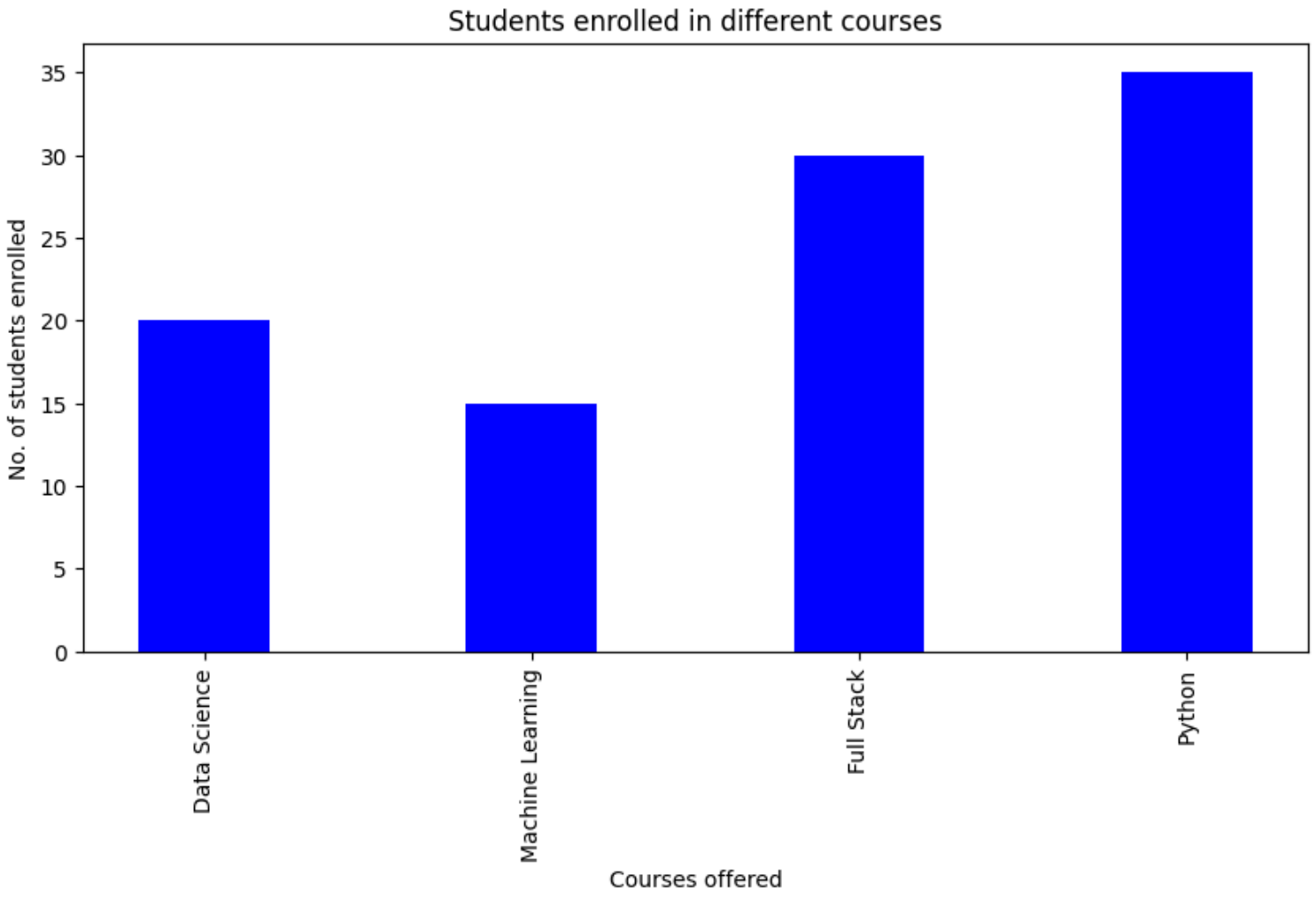
2. Bar Chart

When comparing various categories on the same variable, we use bar plots

[]:

```
data = {'Data Science':20, 'Machine Learning':15, 'Full Stack':30, 'Python':35}
courses = list(data.keys())
values = list(data.values())

#courses = ['Data Science','Machine Learning','Full Stack']

plt.figure(figsize = (10, 5))

# creating the bar plot
plt.bar(courses, values, color ='blue', width = 0.4)

plt.xlabel("Courses offered")
plt.ylabel("No. of students enrolled")
plt.title("Students enrolled in different courses")
plt.xticks(rotation = 90)
plt.show()
```

[]:



3. Scatter Plot

It is effective when we compare one variable across multiple subjects. Ex - How much actual sales happened for some marketing budget.

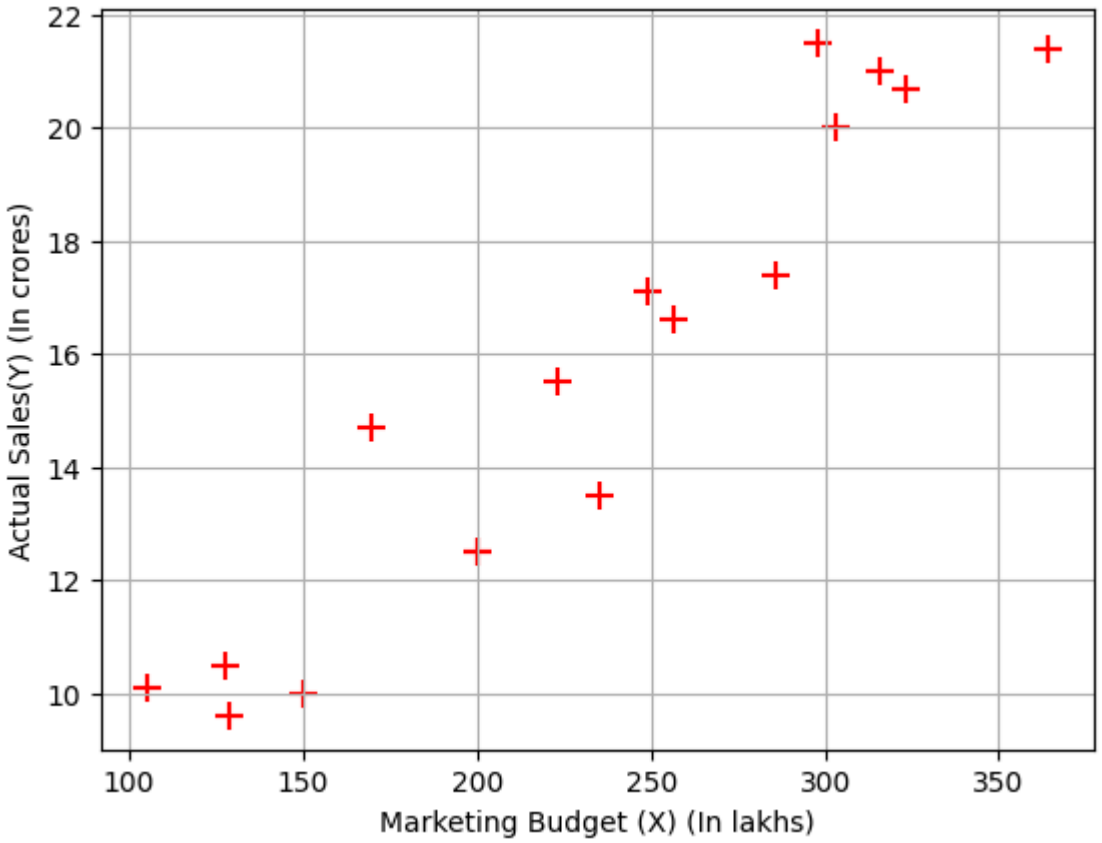
[]:
```
import pandas as pd
from google.colab import files
uploaded = files.upload()
data = pd.read_csv("marketing_data.csv")
data.dropna(inplace = True)
```

[]: Choose files No file chosen — Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

[]:
```
Saving marketing_data.csv to marketing_data.csv
```

[]:
```
X = data['Marketing Budget (X) (In lakhs)']
Y = data['Actual Sales(Y) (In crores)']

plt.scatter(X, Y, color = 'red', s = 100, marker = '+')
plt.xlabel('Marketing Budget (X) (In lakhs)')
plt.ylabel('Actual Sales(Y) (In crores)')
plt.grid(True)
plt.show()
```

[ ]:



5. Histogram

Histogram is the plot used to show the frequency of continous or discrete variable.

To show distribution of the data for a variable, we can use histogram.

[ ]:
```
from google.colab import files
uploaded = files.upload()
market_df = pd.read_csv("market_fact.csv")
market_df.head()
```

[ ]: Choose files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

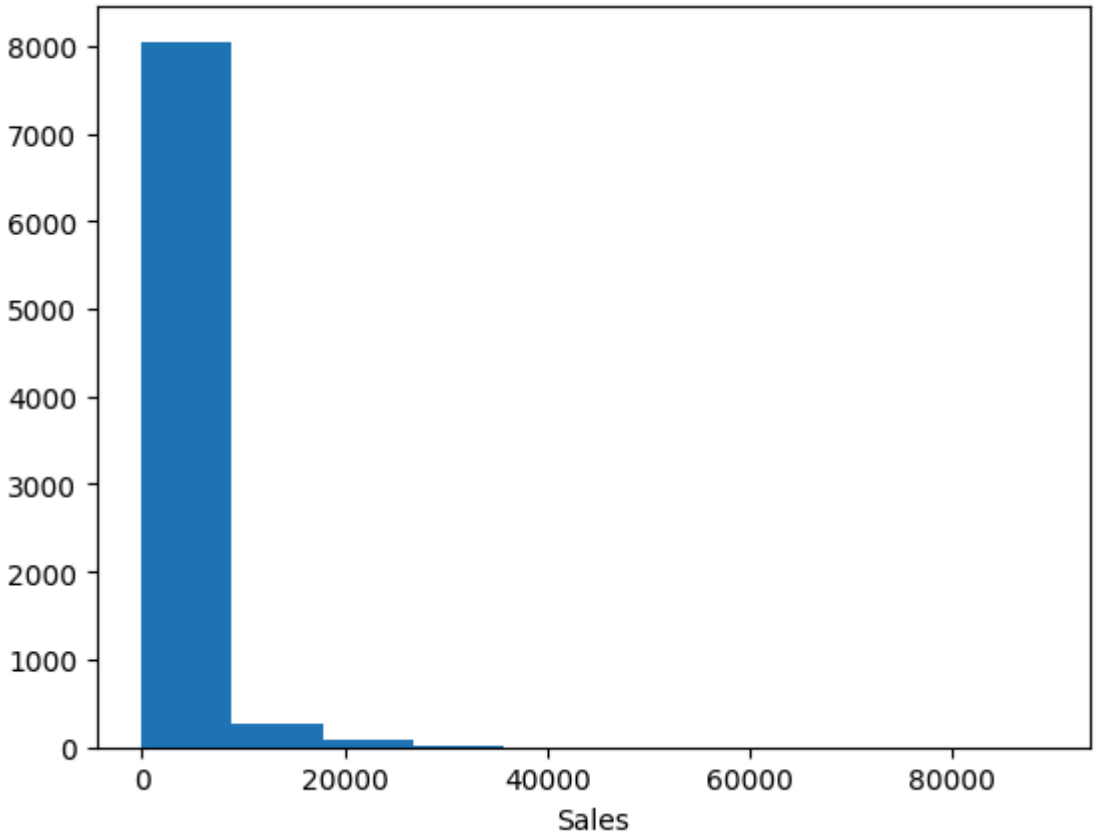
[ ]:
```
Saving market_fact.csv to market_fact.csv
```

[ ]:

	Ord_id	Prod_id	Ship_id	Cust_id	Sales	Discount	Order_Quantity	Profit	Shipping_Cost	Product_Base_Margin
0	Ord_5446	Prod_16	SHP_7609	Cust_1818	136.81	0.01	23	-30.51	3.60	0.56
1	Ord_5406	Prod_13	SHP_7549	Cust_1818	42.27	0.01	13	4.56	0.93	0.54
2	Ord_5446	Prod_4	SHP_7610	Cust_1818	4701.69	0.00	26	1148.90	2.50	0.59
3	Ord_5456	Prod_6	SHP_7625	Cust_1818	2337.89	0.09	43	729.34	14.30	0.37
4	Ord_5485	Prod_17	SHP_7664	Cust_1818	4233.15	0.08	35	1219.87	26.30	0.38

[ ]:
```
plt.hist(market_df['Sales'])
plt.xlabel('Sales')
plt.show()
```

[]:



8000 products gave us profit in range of -100 to 100 dollars

Box Plot

Box plot shows a good summary of the data and also identifies the outliers in the data.

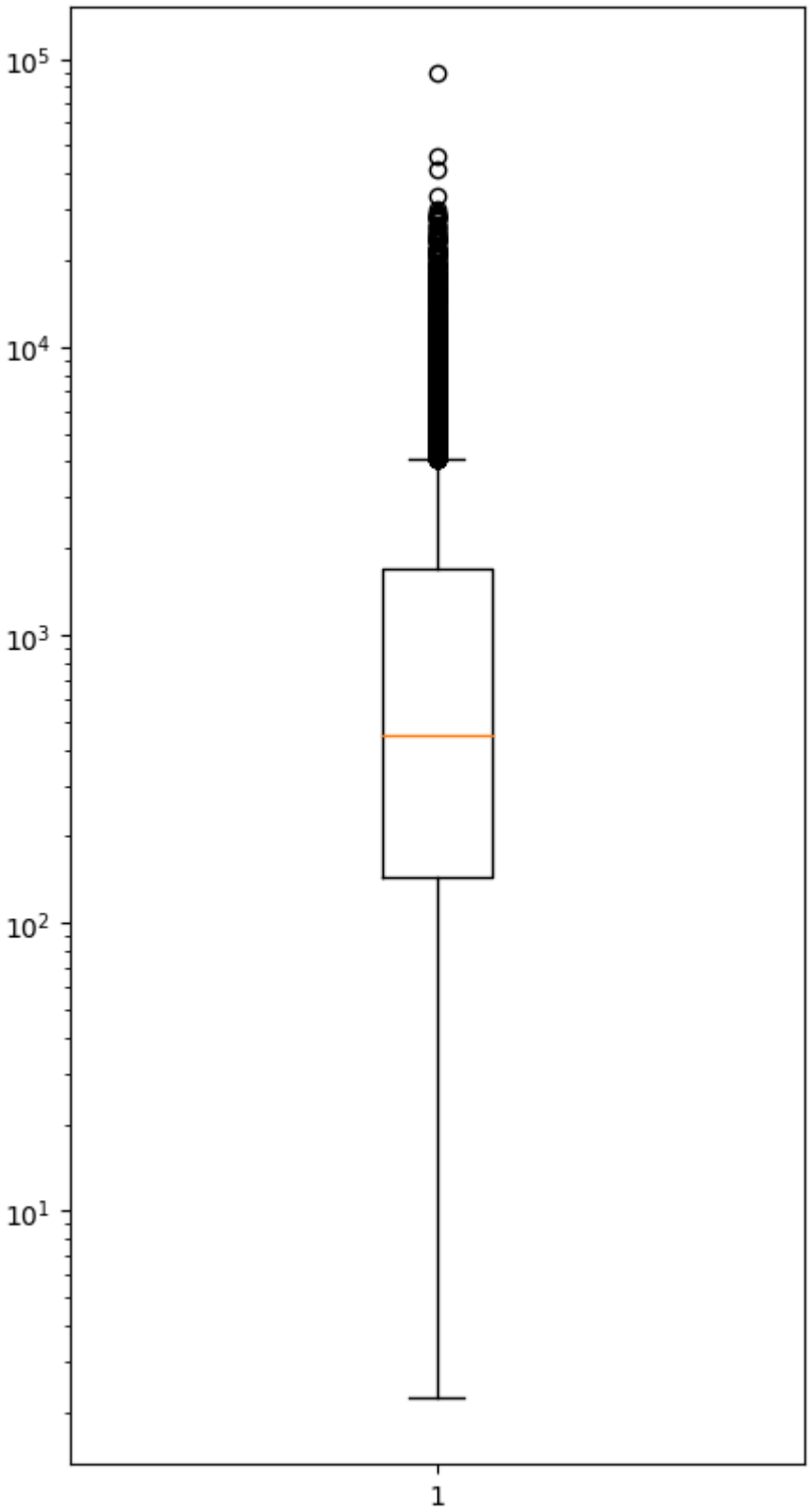
[]:
```
market_df.describe()
```

[]:

	Sales	Discount	Order_Quantity	Profit	Shipping_Cost	Product_Base_Margin
count	8399.000000	8399.000000	8399.000000	8399.000000	8399.000000	8336.000000
mean	1775.878179	0.049671	25.571735	181.184424	12.838557	0.512513
std	3585.050525	0.031823	14.481071	1196.653371	17.264052	0.135589
min	2.240000	0.000000	1.000000	-14140.700000	0.490000	0.350000
25%	143.195000	0.020000	13.000000	-83.315000	3.300000	0.380000
50%	449.420000	0.050000	26.000000	-1.500000	6.070000	0.520000
75%	1709.320000	0.080000	38.000000	162.750000	13.990000	0.590000
max	89061.050000	0.250000	50.000000	27220.690000	164.730000	0.850000

[]:
```
plt.figure(figsize = (5, 10))

plt.boxplot(market_df['Sales'])
plt.yscale('log')
plt.show()
```

[]:



Visualizations with Seaborn

- In the world of Analytics, the best way to get insights is by visualizing the data.
- We have already used Matplotlib, a 2D plotting library that allows us to plot different graphs and charts.
- Another complimentary package that is based on this data visualization library is Seaborn, which provides a high-level interface to draw statistical graphics.

Seaborn

- Is a python data visualization library for statistical plotting
- Is based on matplotlib (built on top of matplotlib)
- Is designed to work with NumPy and pandas data structures
- Provides a high-level interface for drawing attractive and informative statistical graphics.
- Comes equipped with preset styles and color palettes so you can create complex, aesthetically pleasing charts with a few lines of code.
- Seaborn is built on top of Python's core visualization library matplotlib, but it's meant to serve as a complement, not a replacement.
- In most cases, we'll still use matplotlib for simple plotting
- Seaborn helps resolve the two major problems faced by Matplotlib, the problems are –
    - Default Matplotlib parameters
    - Working with dataframes

[]:
```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
```

[]:
```
import seaborn as sns
```

We are going to use 2 built in functions of Seaborn to make our visualizations :

- load_dataset()
- get_dataset_names()

[]:
```
print(sns.get_dataset_names())
```

[]:
```
['anagrams', 'anscombe', 'attention', 'brain_networks', 'car_crashes', 'diamonds', 'dots', 'dowjones', 'exercise', 'flights', 'fmri', 'geyser', 'glue',
'healthexp', 'iris', 'mpg', 'penguins', 'planets', 'seaice', 'taxis', 'tips', 'titanic']
```

[]:
```
# we will have a titanic.csv file
titanic = pd.read_csv("titanic.csv")
```

[]:
```
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
<ipython-input-55-1ae22b972f41> in <cell line: 2>()
      1 # we will have a titanic.csv file
----> 2 titanic = pd.read_csv("titanic.csv")

/usr/local/lib/python3.10/dist-packages/pandas/util/_decorators.py in wrapper(*args, **kwargs)
    209                 else:
    210                     kwargs[new_arg_name] = new_arg_value
--> 211             return func(*args, **kwargs)
    212
    213         return cast(F, wrapper)

/usr/local/lib/python3.10/dist-packages/pandas/util/_decorators.py in wrapper(*args, **kwargs)
    329                     stacklevel=find_stack_level(),
    330                 )
--> 331             return func(*args, **kwargs)
    332
    333         # error: "Callable[[VarArg(Any), KwArg(Any)], Any]" has no

/usr/local/lib/python3.10/dist-packages/pandas/io/parsers/readers.py in read_csv(filepath_or_buffer, sep, delimiter, header, names, index_col, usecols, squee
    948     kwds.update(kwds_defaults)
    949
--> 950     return _read(filepath_or_buffer, kwds)
    951
    952


/usr/local/lib/python3.10/dist-packages/pandas/io/parsers/readers.py in _read(filepath_or_buffer, kwds)
    603
    604     # Create the parser.
--> 605     parser = TextFileReader(filepath_or_buffer, **kwds)
    606
    607     if chunksize or iterator:

/usr/local/lib/python3.10/dist-packages/pandas/io/parsers/readers.py in __init__(self, f, engine, **kwds)
   1440
   1441         self.handles: IOHandles | None = None
-> 1442         self._engine = self._make_engine(f, self.engine)
   1443
   1444     def close(self) -> None:

/usr/local/lib/python3.10/dist-packages/pandas/io/parsers/readers.py in _make_engine(self, f, engine)
   1733                 if "b" not in mode:
   1734                     mode += "b"
-> 1735             self.handles = get_handle(
   1736                 f,
   1737                 mode,

/usr/local/lib/python3.10/dist-packages/pandas/io/common.py in get_handle(path_or_buf, mode, encoding, compression, memory_map, is_text, errors, storage_opt
    854         if ioargs.encoding and "b" not in ioargs.mode:
    855             # Encoding
--> 856             handle = open(
    857                 handle,
    858                 ioargs.mode,

FileNotFoundError: [Errno 2] No such file or directory: 'titanic.csv'
```

Data visualization using Seaborn

- Visualizing statistical relationships
- Plotting categorical data
- Distribution of dataset
- Visualizing Linear Relationships

Visualizing Statistical Relationship

The process of understanding relationships between variables of a dataset and how these relationships, in turn, depend on other variables is known as statistical analysis

- relplot()
  - This is a figure-level-function that makes use of two other axes functions for Visualizing Statistical Relationships which are -
    - scatterplot()
    - lineplot()
  - By default it plots scatterplot()

Scatter Plot

- A scatterplot is the most common example of visualizing relationships between two variables.
- It depicts the joint distribution of two variables using a cloud of points, where each point represents an observation in the dataset.
- This depiction allows the eye to infer a substantial amount of information about whether there is any meaningful relationship between them.

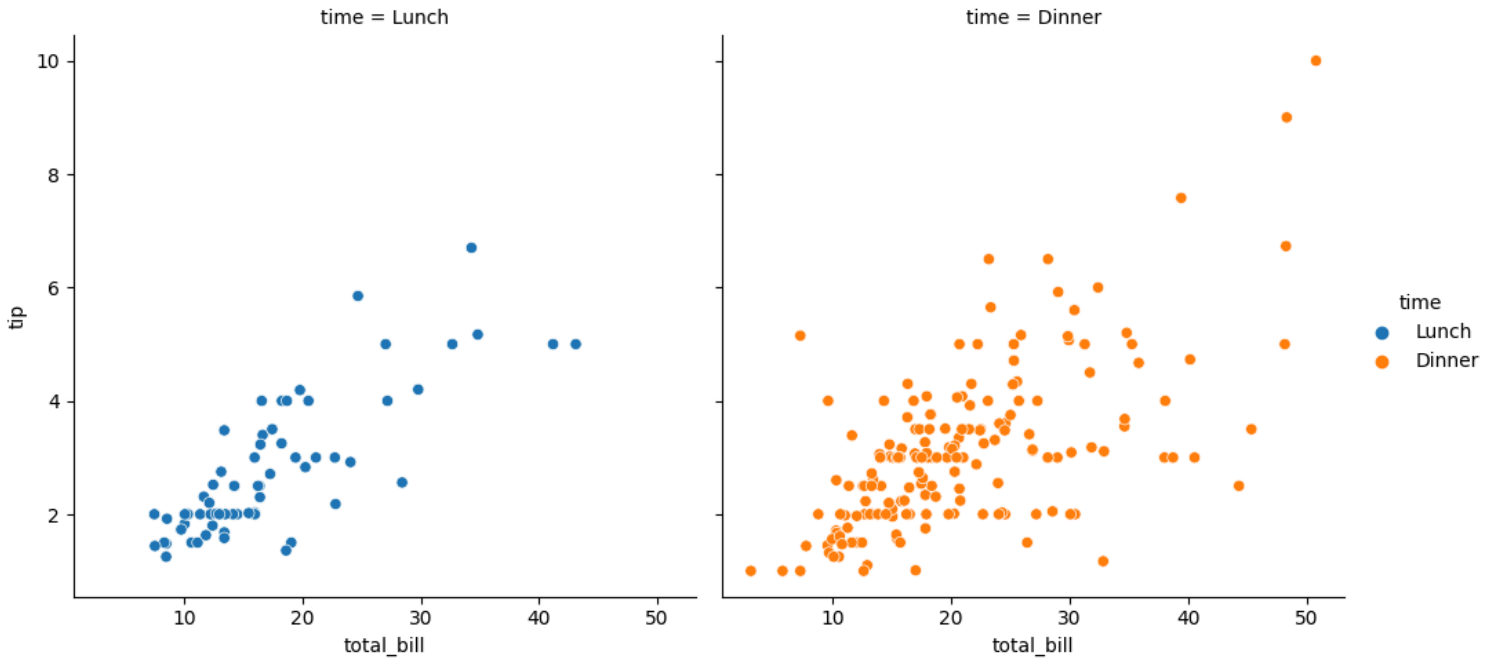
[]:
```
df = sns.load_dataset('tips')
df.head()
```

[]:

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

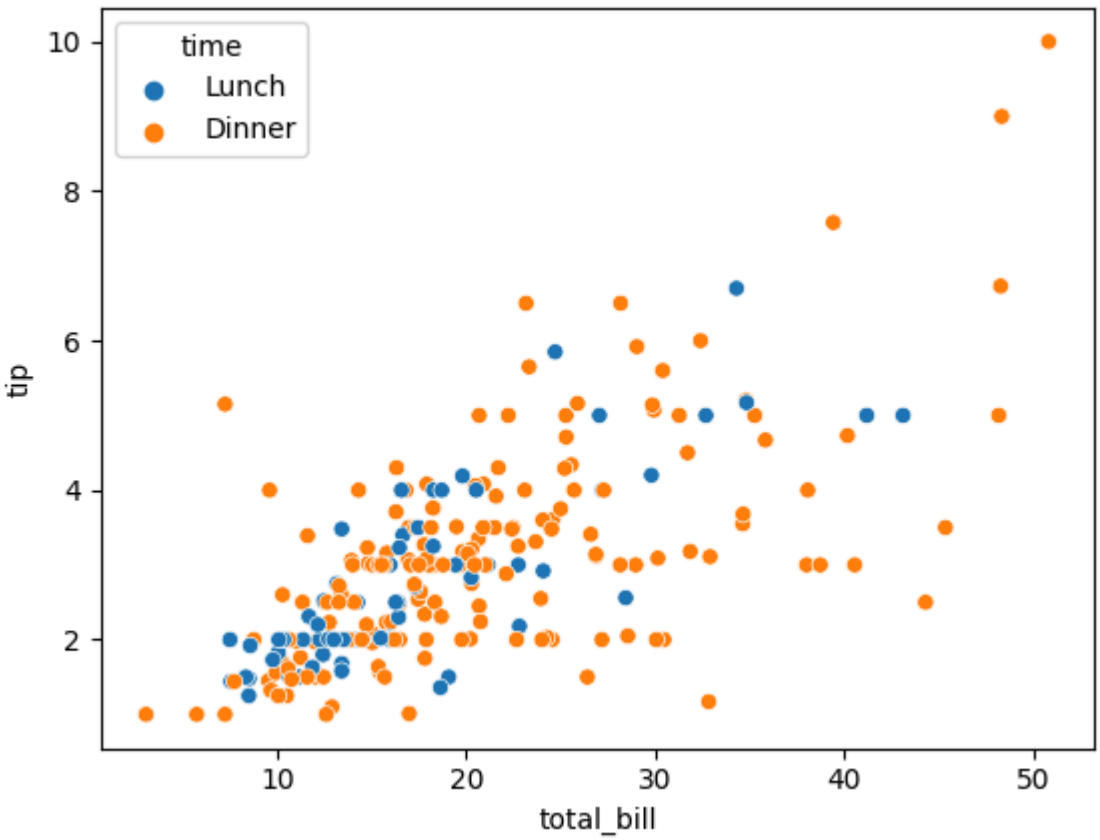
[]:
```
sns.relplot(x = 'total_bill', y='tip', data = df, col='time', hue='time', kind= 'scatter')
plt.show()
```

[]:



[]:
```
sns.scatterplot(x = 'total_bill', y='tip', data = df, hue='time')
plt.show()
```

[]:

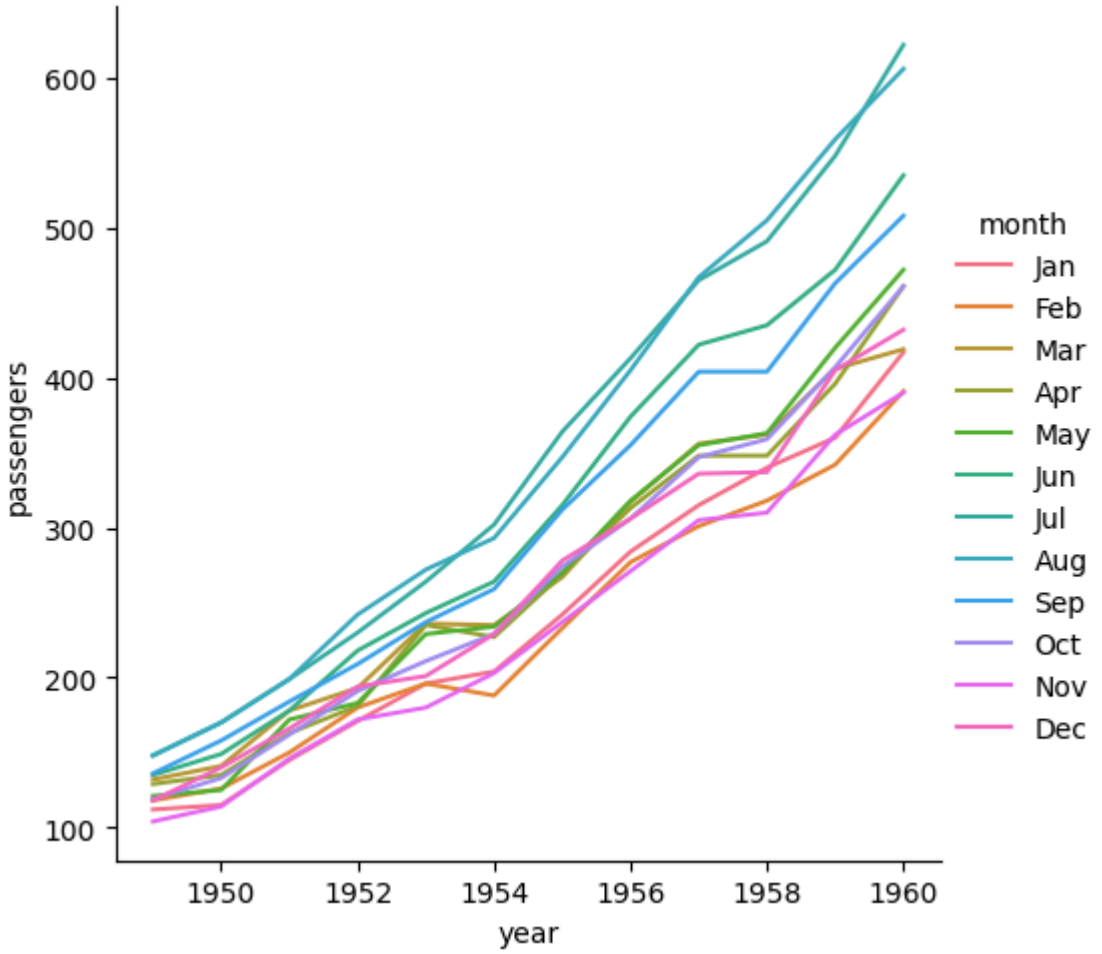


Line Plot

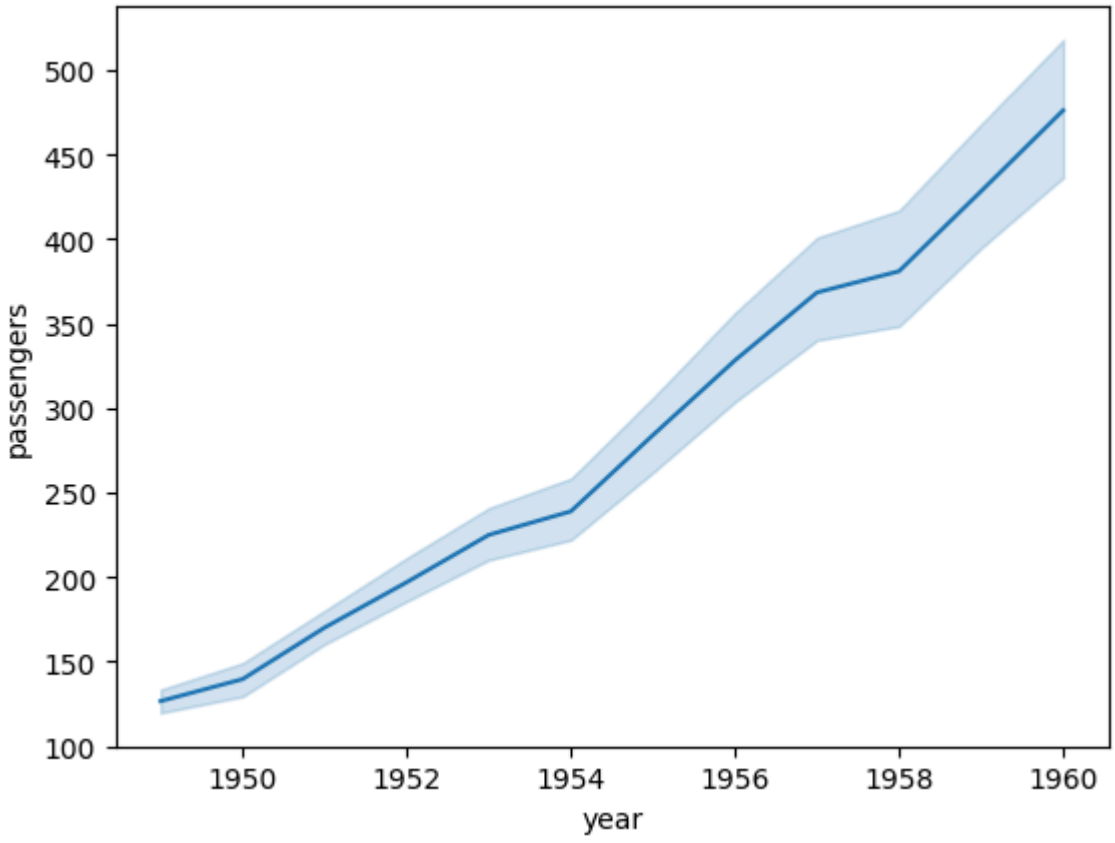
- Suitable when we want to understand change in one variable as a function of time or some other continuous variable.
- It depicts the relationship between continuous as well as categorical values in a continuous data point format.

[]:
```
flights = sns.load_dataset('flights')
```

[]:
```
sns.relplot(x='year', y='passengers', data=flights, kind='line', hue='month')
plt.show()
```

[]:


[]:
```
sns.lineplot(x='year', y='passengers',data=flights)
plt.show()
```

[]:


Categorical Data Plotting

- Now we'll see the relationship between two variables of which one would be categorical.
- There are several ways to visualize a relationship involving categorical data in seaborn.
- catplot() - it gives a unified approach to plot different kind of categorical plots in seaborn

Categorical Distribution Plots

- As the size of the dataset grows, categorical scatter plots become limited in the information they can provide about the distribution of values within each category.
- In such cases we can use several approaches for summarizing the distributional information in ways that facilitate easy comparisons across the category levels.
- We use distribution plots to get a sense for how the variables are distributed.

Box Plot

- It is also named a box-and-whisker plot.
- Boxplots are used to visualize distributions. This is very useful when we want to compare data between two groups.
- Box Plot is the visual representation of the depicting groups of numerical data through their quartiles.
- Boxplot is also used to detect the outliers in data set.

 boxplot.jpg

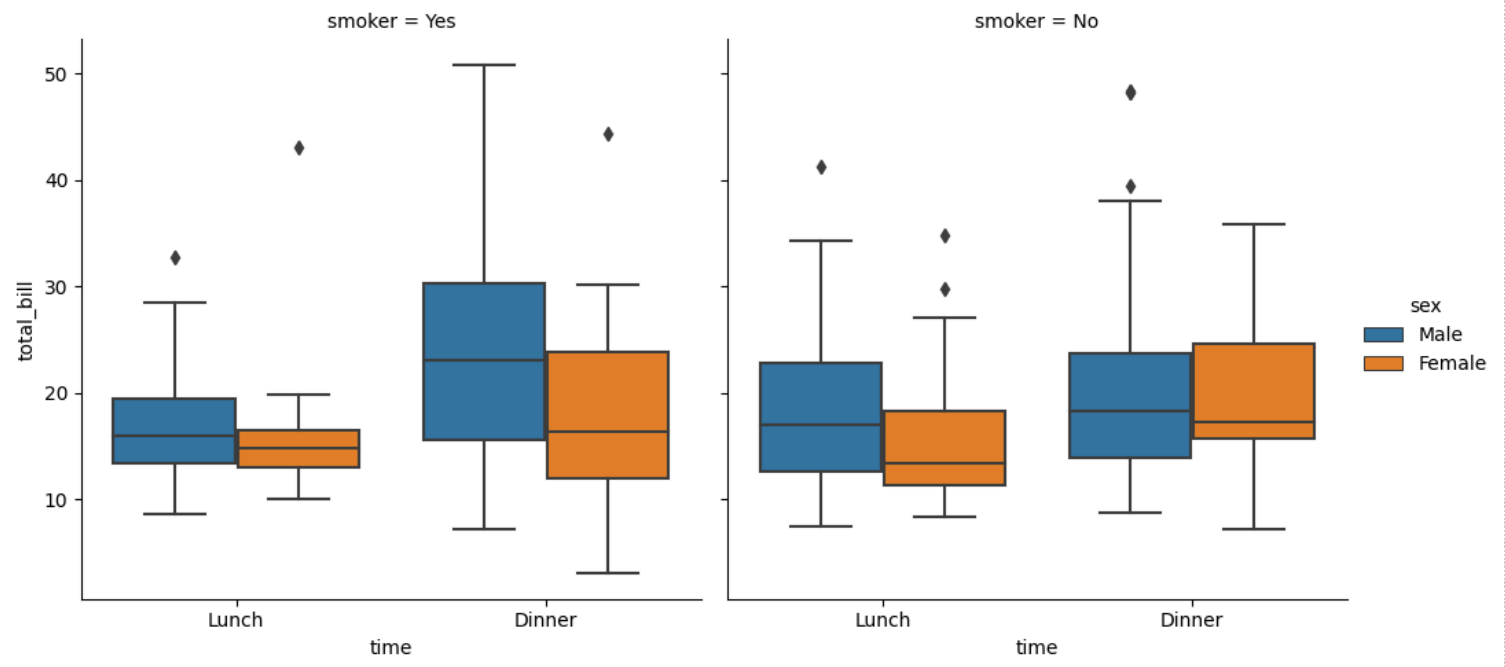
- It captures the summary of the data efficiently with a simple box and whiskers and allows us to compare easily across groups.
- Boxplot summarizes a sample data using 25th, 50th and 75th percentiles. These percentiles are also known as the lower quartile, median and upper quartile.
- A box plot consist of 5 things.
    - Minimum
    - First Quartile or 25%
    - Median (Second Quartile) or 50%
    - Third Quartile or 75%
    - Maximum

[ ]:
```
tips=sns.load_dataset('tips')
tips.head()
```

[ ]:

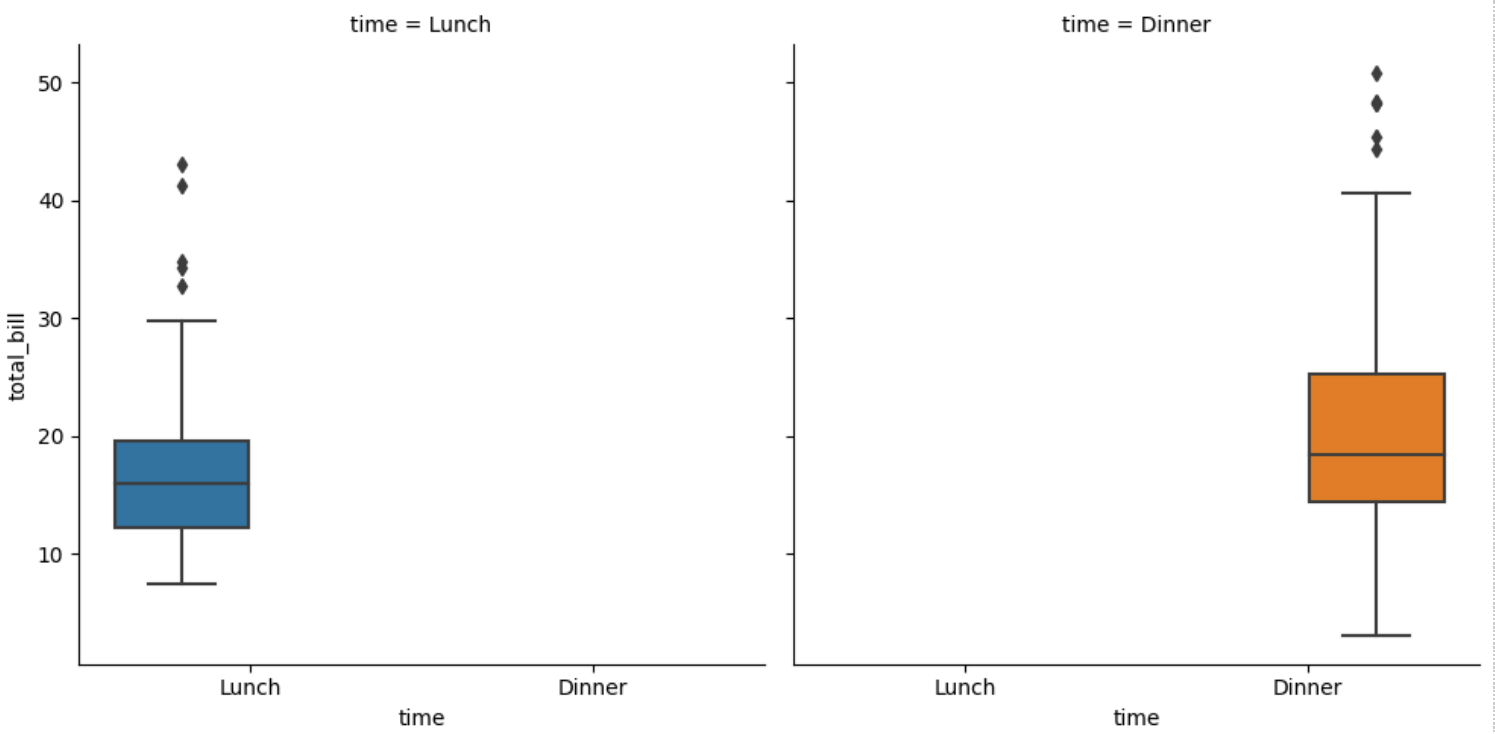
	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

[ ]:
```
sns.catplot(x = 'time', y = 'total_bill',data =tips, kind= 'box', hue = 'sex', col = 'smoker')
plt.show()
```

[ ]:


[ ]:
```
sns.catplot(x='time', y='total_bill', data=df, kind='box', hue='time', col='time')
plt.show()
```

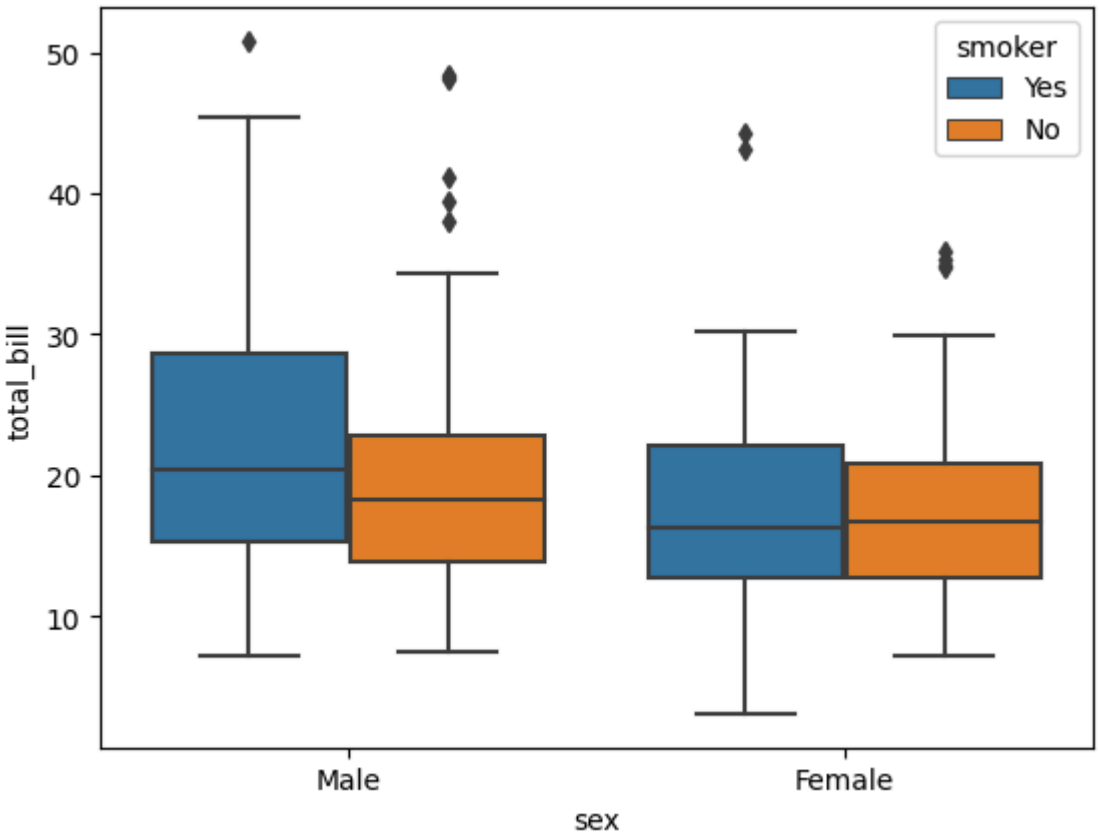
[]:



[]:

```
sns.boxplot(x='sex', y='total_bill', data=df, hue='smoker')
plt.show()
```

[]:



Violin Plot

- It shows the distribution of quantitative data across several levels of one (or more) categorical variables such that those distributions can be compared.
- It is really close from a boxplot, but allows a deeper understanding of the density.


1_TTMOaNG1o4PgQd-e8LurMg.png

- The density is mirrored and flipped over and the resulting shape is filled in, creating an image resembling a violin.
- The advantage of a violin plot is that it can show nuances in the distribution that aren't perceptible in a boxplot.
- On the other hand, the boxplot more clearly shows the outliers in the data.
- Violins are particularly adapted when the amount of data is huge and showing individual observations gets impossible.

[]:

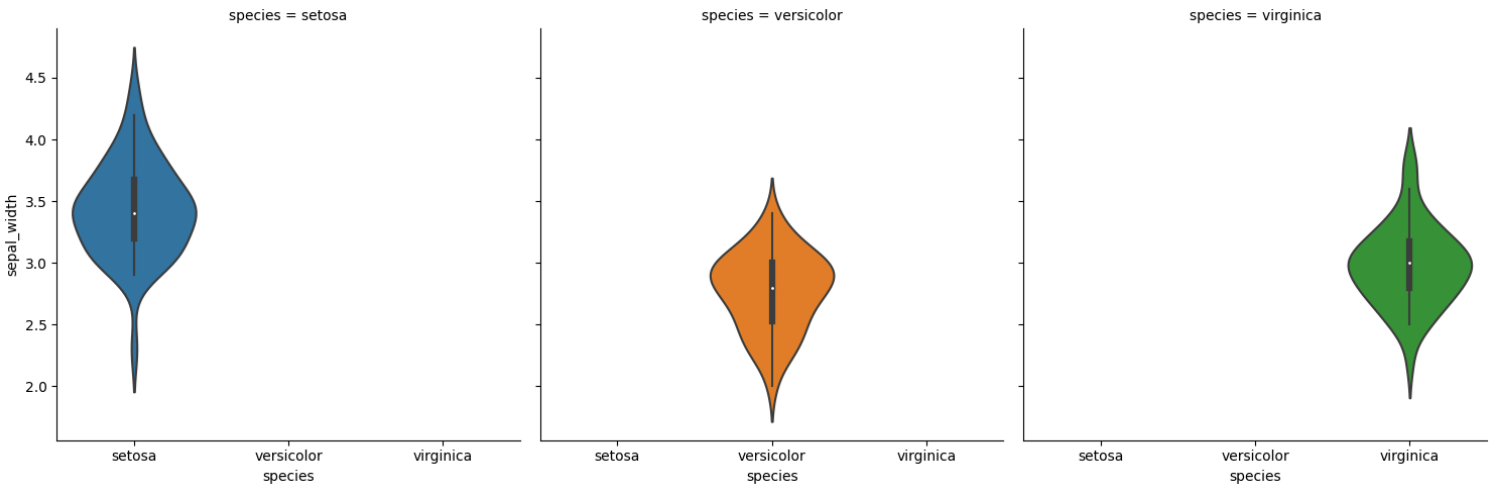
```
iris=sns.load_dataset('iris')
iris.head()
```

[]:

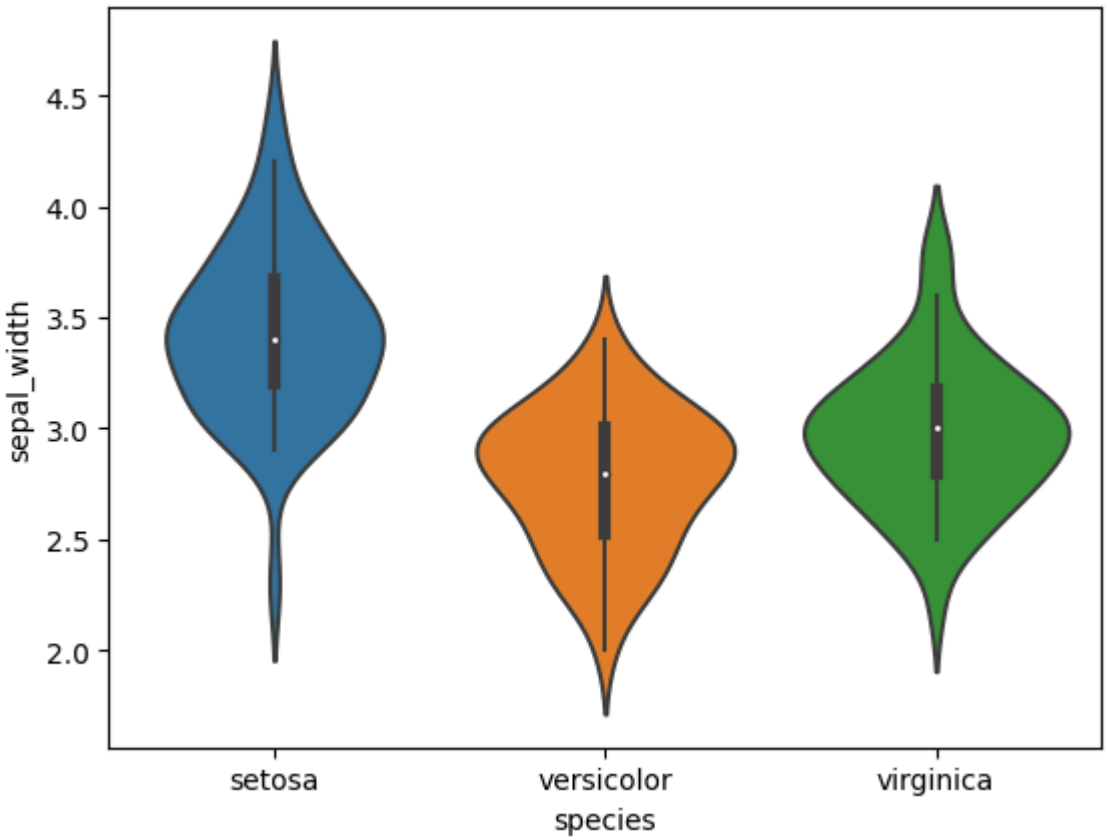
	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa

	sepal_length	sepal_width	petal_length	petal_width	species
4	5.0	3.6	1.4	0.2	setosa

[ ]:
```
sns.catplot(x = 'species', y = 'sepal_width', data = iris, kind='violin', col = 'species')
plt.show()
```

[ ]:


[ ]:
```
sns.violinplot(x='species', y='sepal_width', data = iris)
plt.show()
```

[ ]:


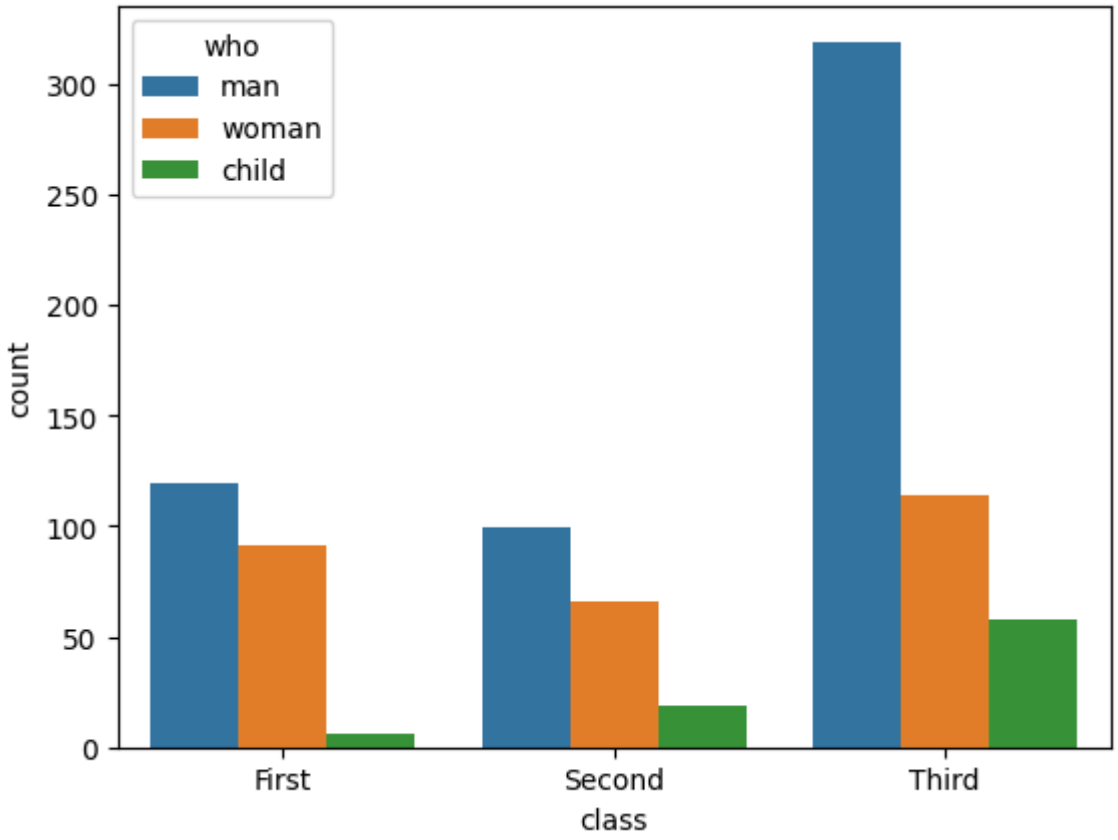
Countplots

They show the count of observations in each column mentioned using a bar chart. It is like a histogram but for categorical variables.

[ ]:
```
titanic = sns.load_dataset('titanic')
```

[ ]:
```
sns.countplot(x = 'class', data = titanic, hue='who')
plt.show()
```

[]:



Visualizing Data Distributions

Distplot

Plots a univariate distribution of observations

[]:
```
market_df.describe()
```

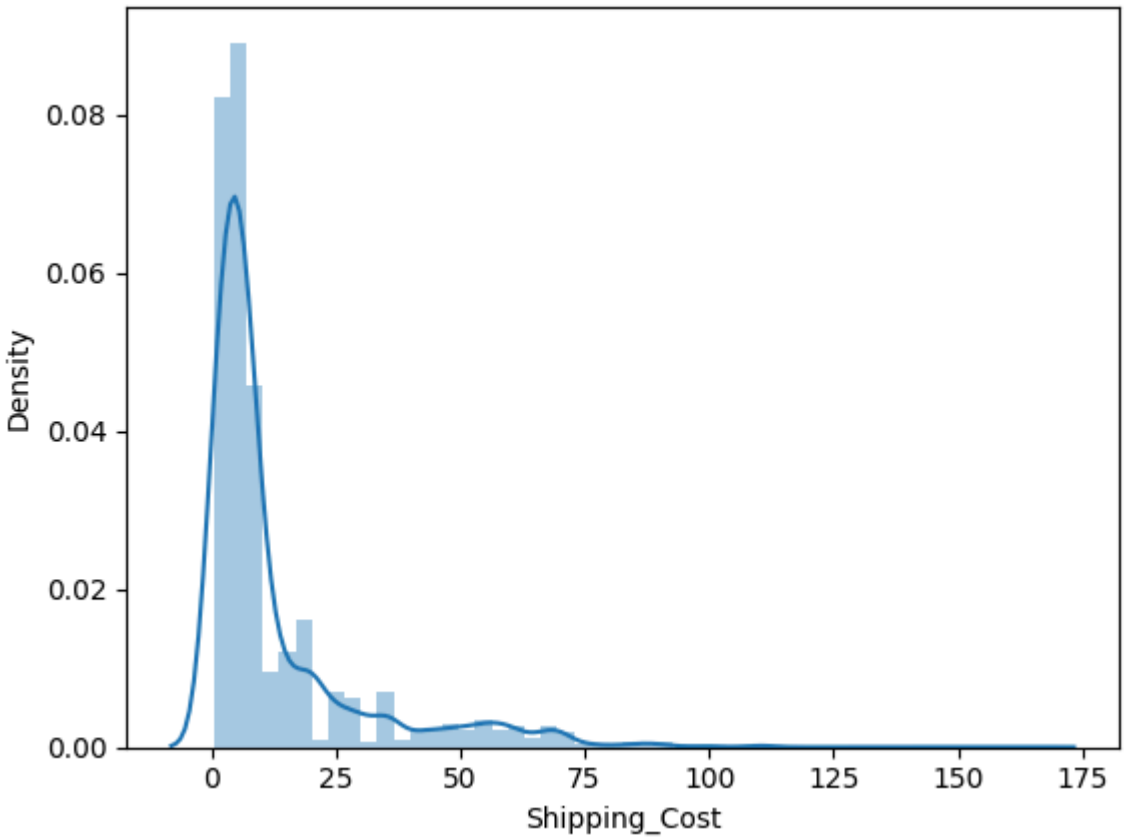
[]:

	Sales	Discount	Order_Quantity	Profit	Shipping_Cost	Product_Base_Margin
count	8399.000000	8399.000000	8399.000000	8399.000000	8399.000000	8336.000000
mean	1775.878179	0.049671	25.571735	181.184424	12.838557	0.512513
std	3585.050525	0.031823	14.481071	1196.653371	17.264052	0.135589
min	2.240000	0.000000	1.000000	-14140.700000	0.490000	0.350000
25%	143.195000	0.020000	13.000000	-83.315000	3.300000	0.380000
50%	449.420000	0.050000	26.000000	-1.500000	6.070000	0.520000
75%	1709.320000	0.080000	38.000000	162.750000	13.990000	0.590000
max	89061.050000	0.250000	50.000000	27220.690000	164.730000	0.850000

[]:
```
sns.distplot(market_df['Shipping_Cost'])
plt.show()
```

[]:
```
<ipython-input-78-addbf75431ba>:1: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(market_df['Shipping_Cost'])
```

[ ]:



[ ]:
```
import warnings
warnings.filterwarnings('ignore')
```

Jointplot

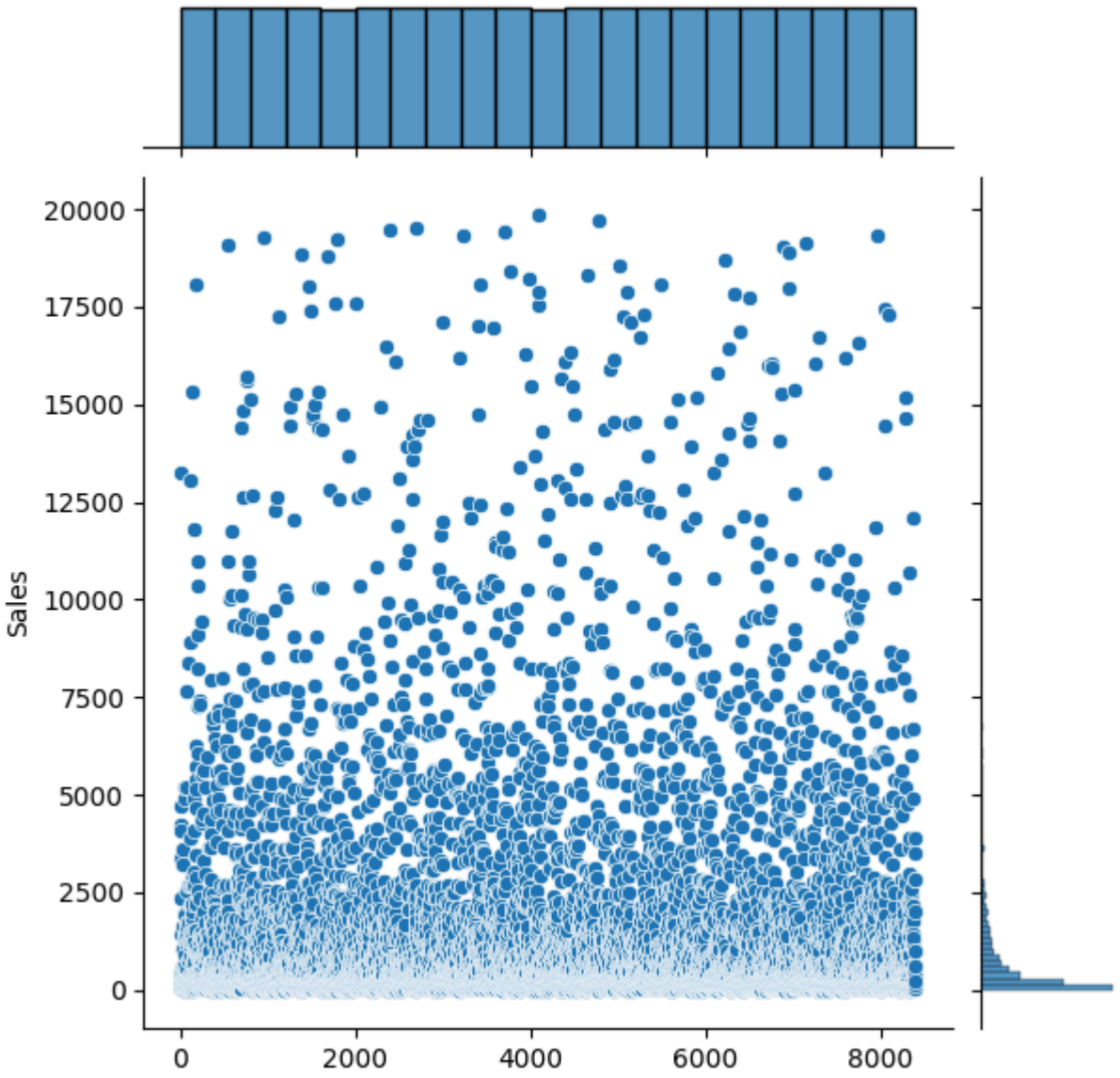
Used to plot two variables with Bivariate scatterplot and Univariate Histogram in the same plot.

[ ]:
```
sns.jointplot(market_df['Sales'], market_df['Profit'])
plt.show()
```

[ ]:
```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-82-d0fed3c79442> in <cell line: 1>()
----> 1 sns.jointplot(market_df['Sales'], market_df['Profit'])
      2 plt.show()

TypeError: jointplot() takes from 0 to 1 positional arguments but 2 were given
```

[ ]:
```
sns.jointplot(market_df['Sales'])
plt.show()
```

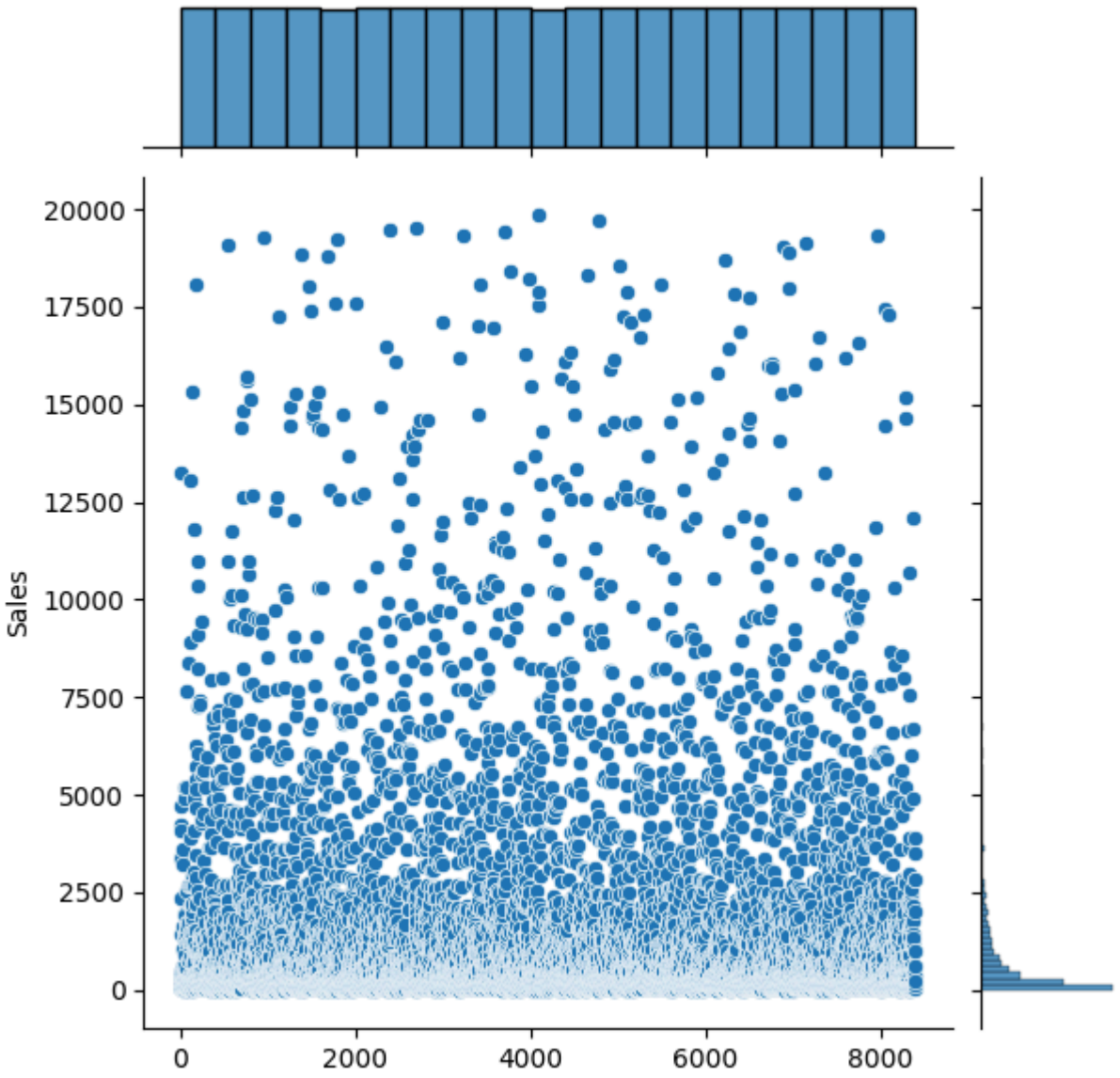
[]:



[]:

```
# remove points having extreme values
market_df = market_df[(market_df.Profit < 10000) & (market_df.Sales < 20000)]

sns.jointplot(market_df['Sales'])
plt.show()
```

[]:



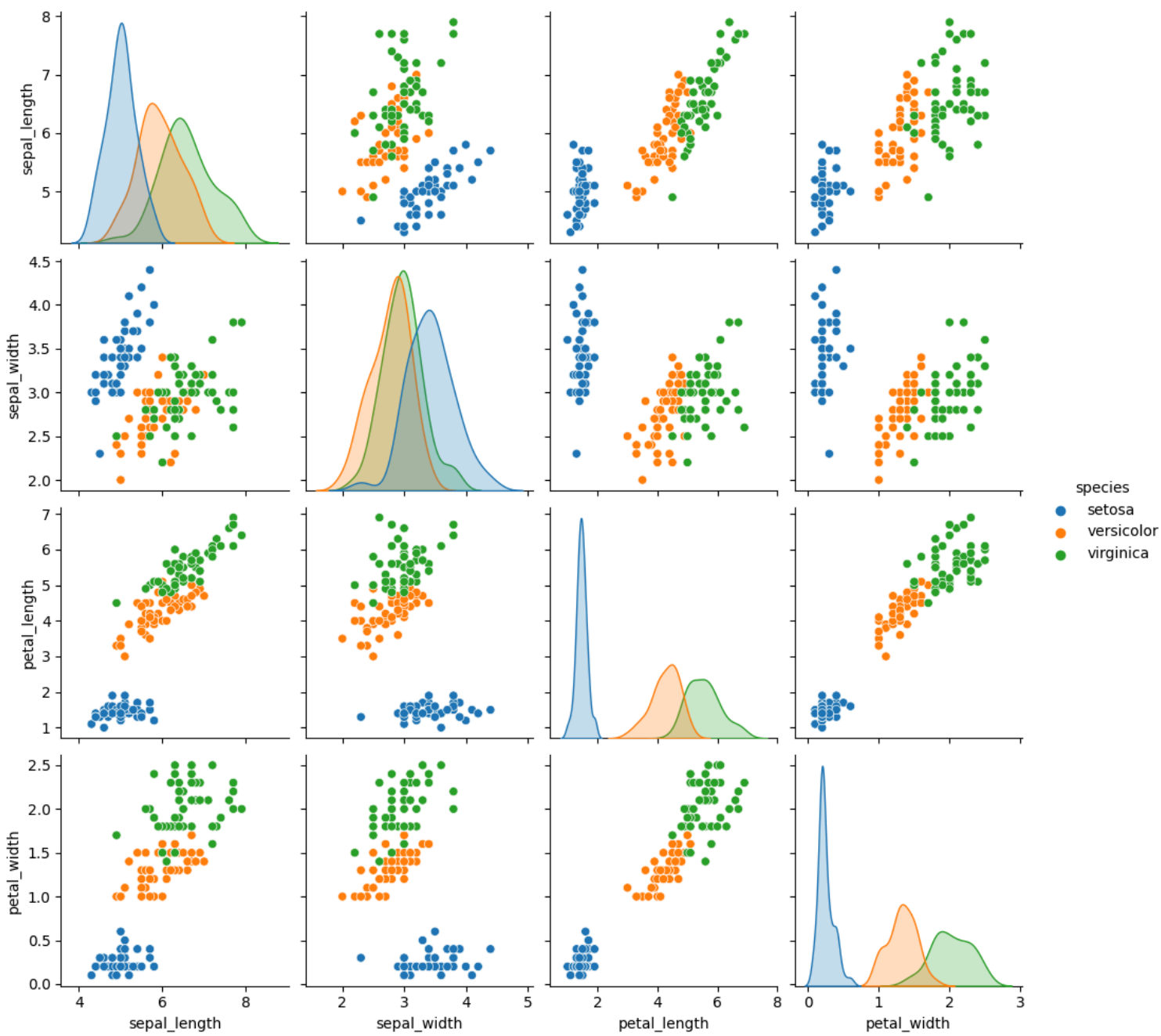
Pairplots

To plot multiple pairwise bivariate distributions in a dataset, you can use the pairplot() function.

[]:
```
iris = sns.load_dataset('iris')
```

[]:
```
sns.pairplot(iris, hue='species')
plt.show()
```

[]:



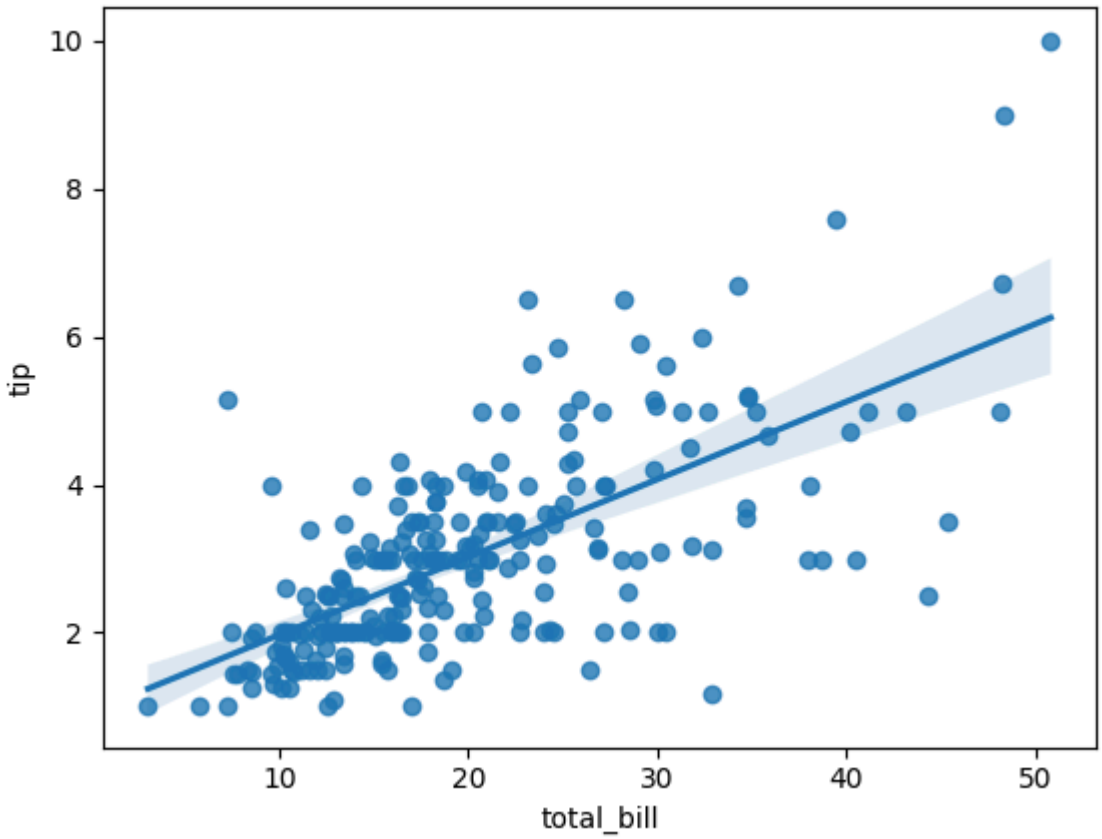
Regplot

Plot data and a linear regression model fit.

[]:
```
sns.regplot(x = 'total_bill', y = 'tip', data = tips)
```

[]:

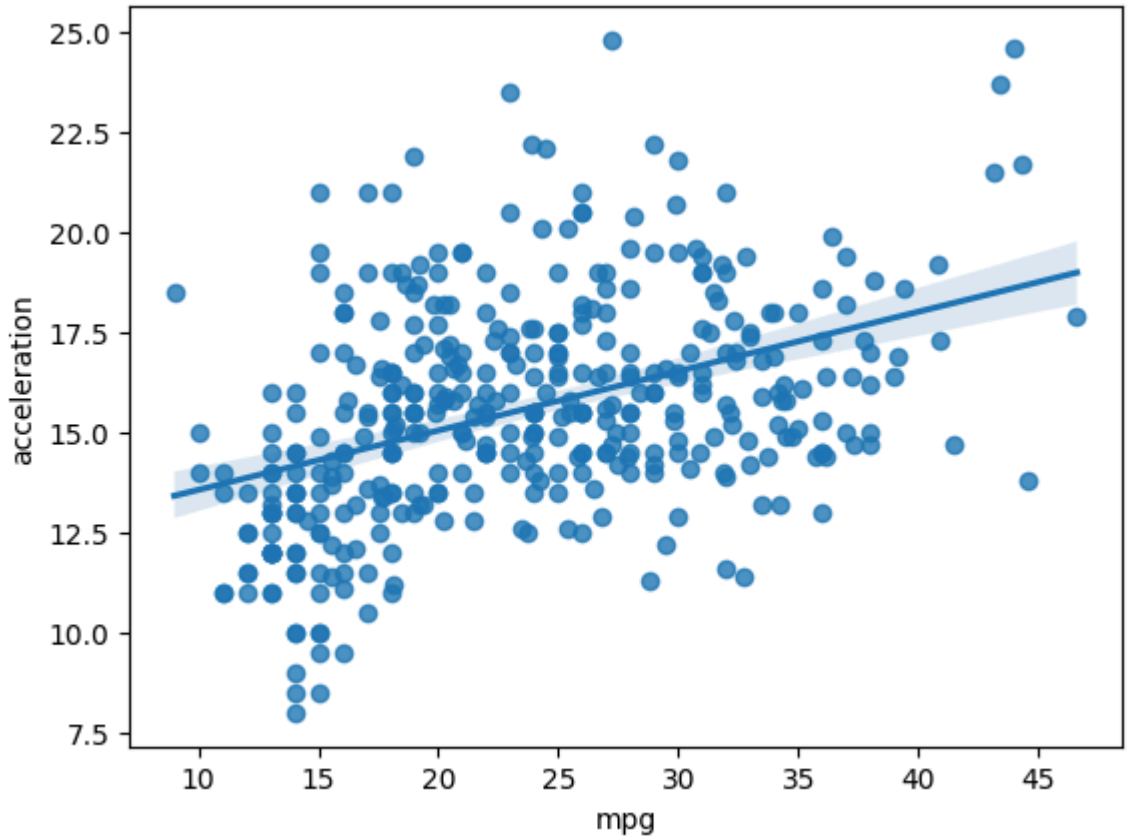
[]:



[]:

```
data = sns.load_dataset('mpg')

sns.regplot(x='mpg', y='acceleration', data = data)
plt.show()
```

[]:



Density Chart

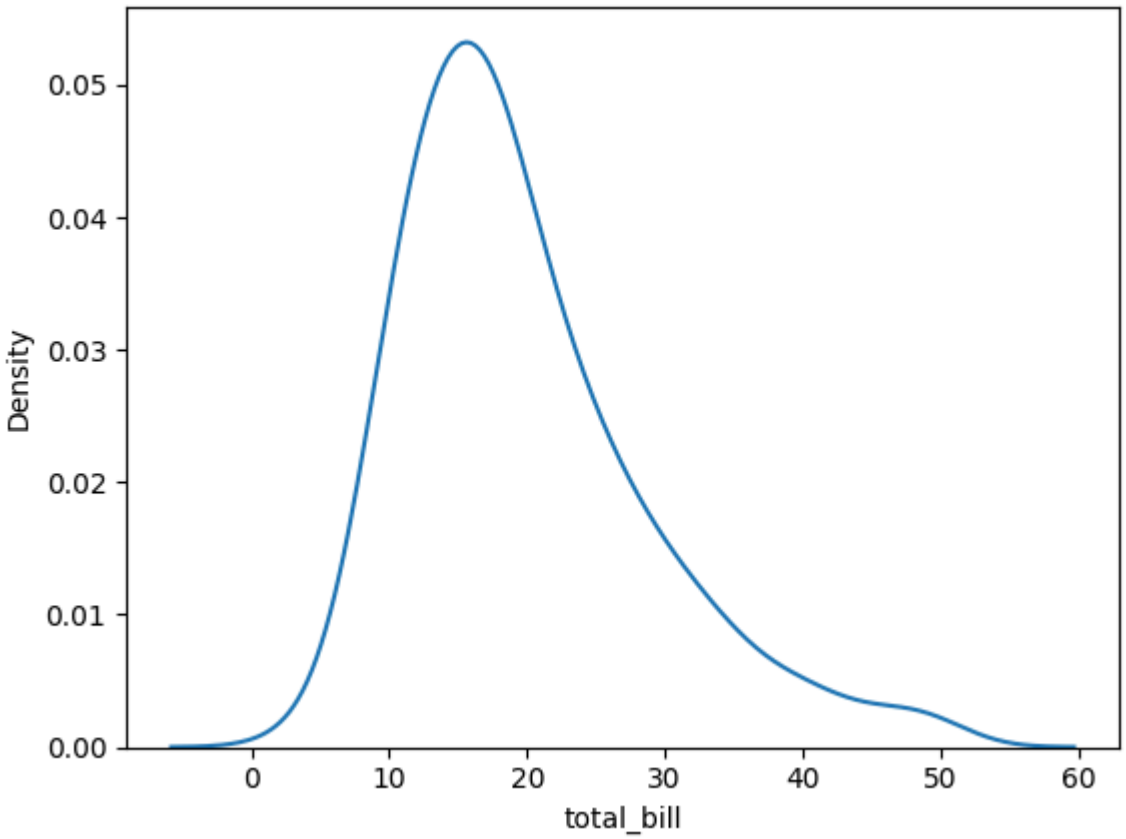
A density plot is a representation of the distribution of a numeric variable. Density plots are used to study the distribution of one or a few variables.

[]:

```
sns.kdeplot(data=tips, x="total_bill")
```

[]:

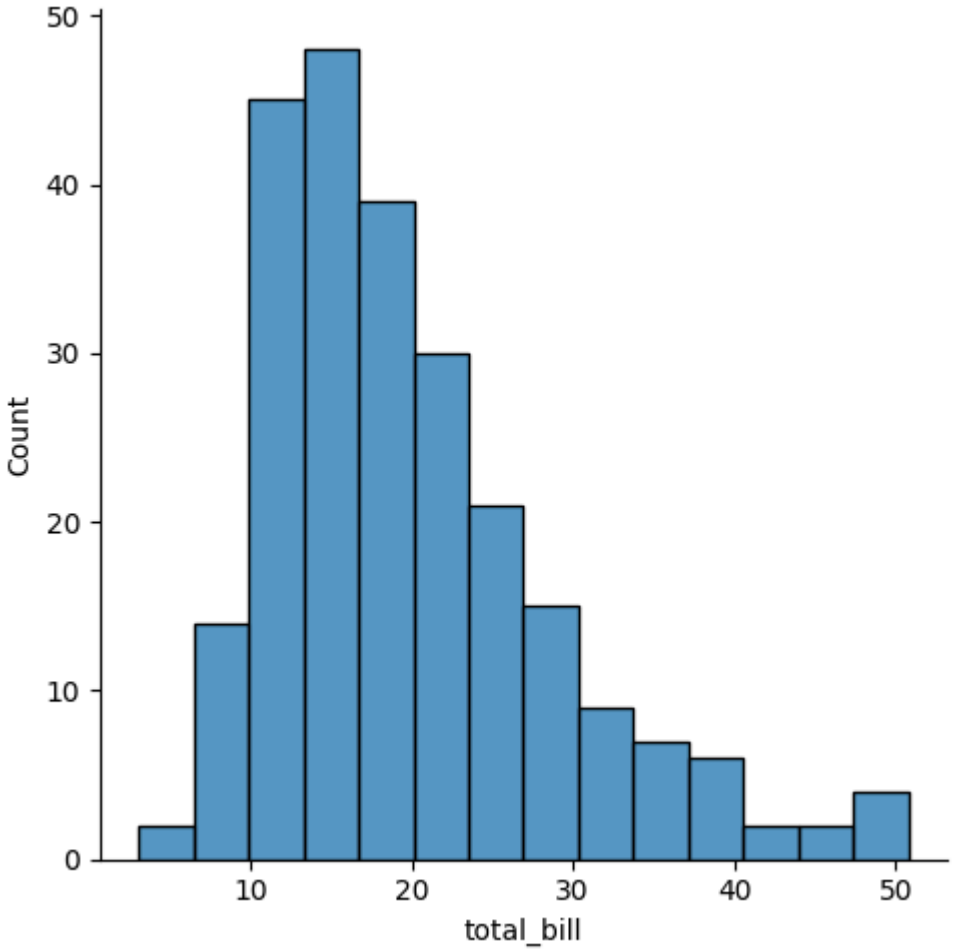
[]:



[]:

```
sns.displot(x = 'total_bill', data = tips)
plt.show()
```

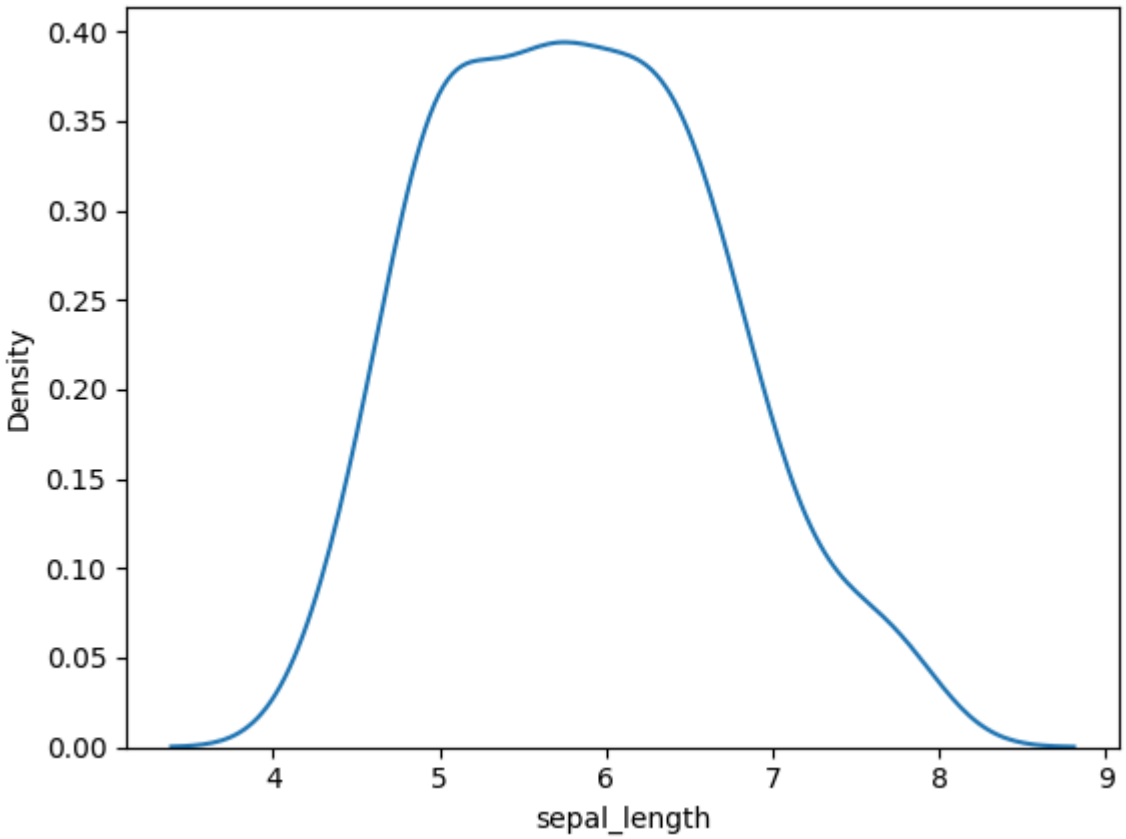
[]:



[]:

```
sns.kdeplot(x='sepal_length', data=iris)
plt.show()
```

[]:



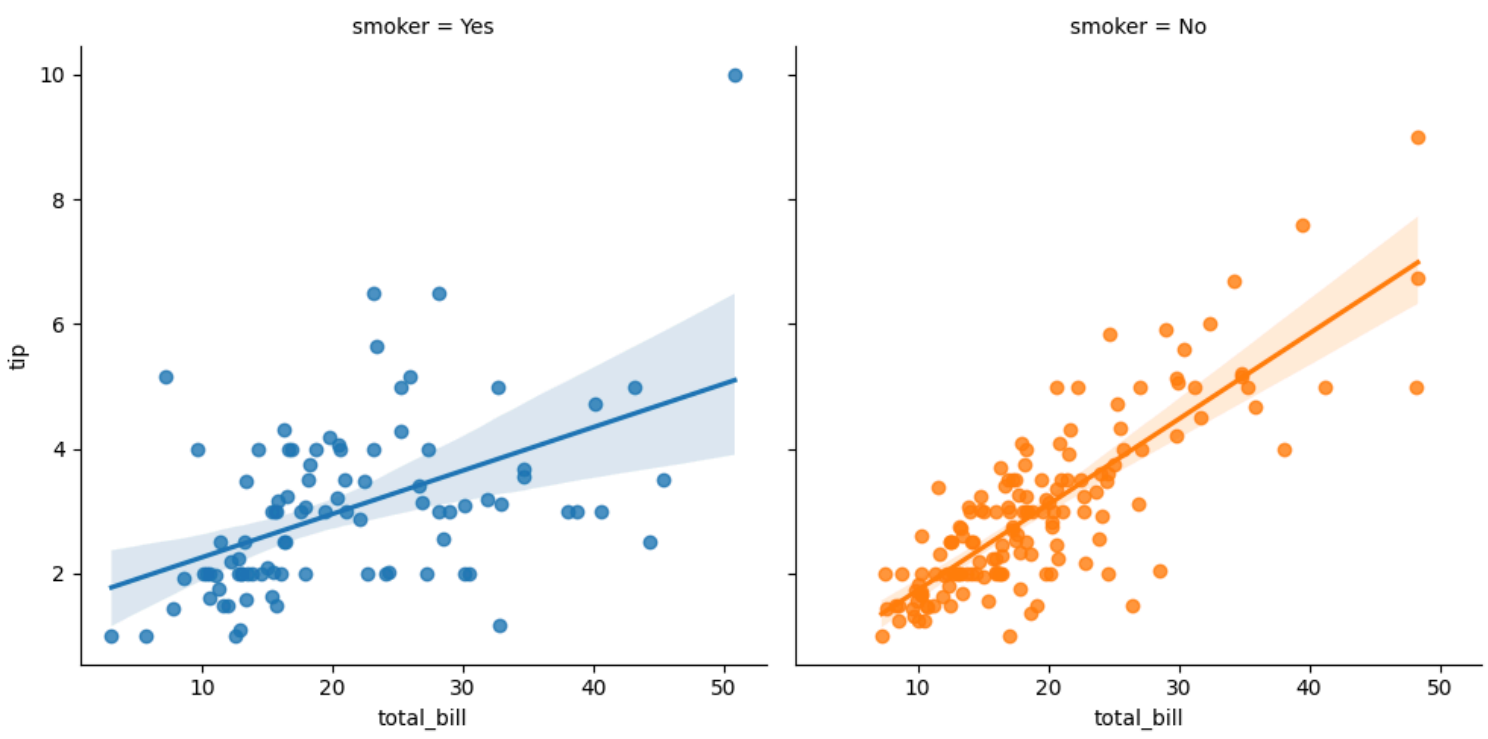
lmplot

Plot data and regression model fits across a FacetGrid.

[]:

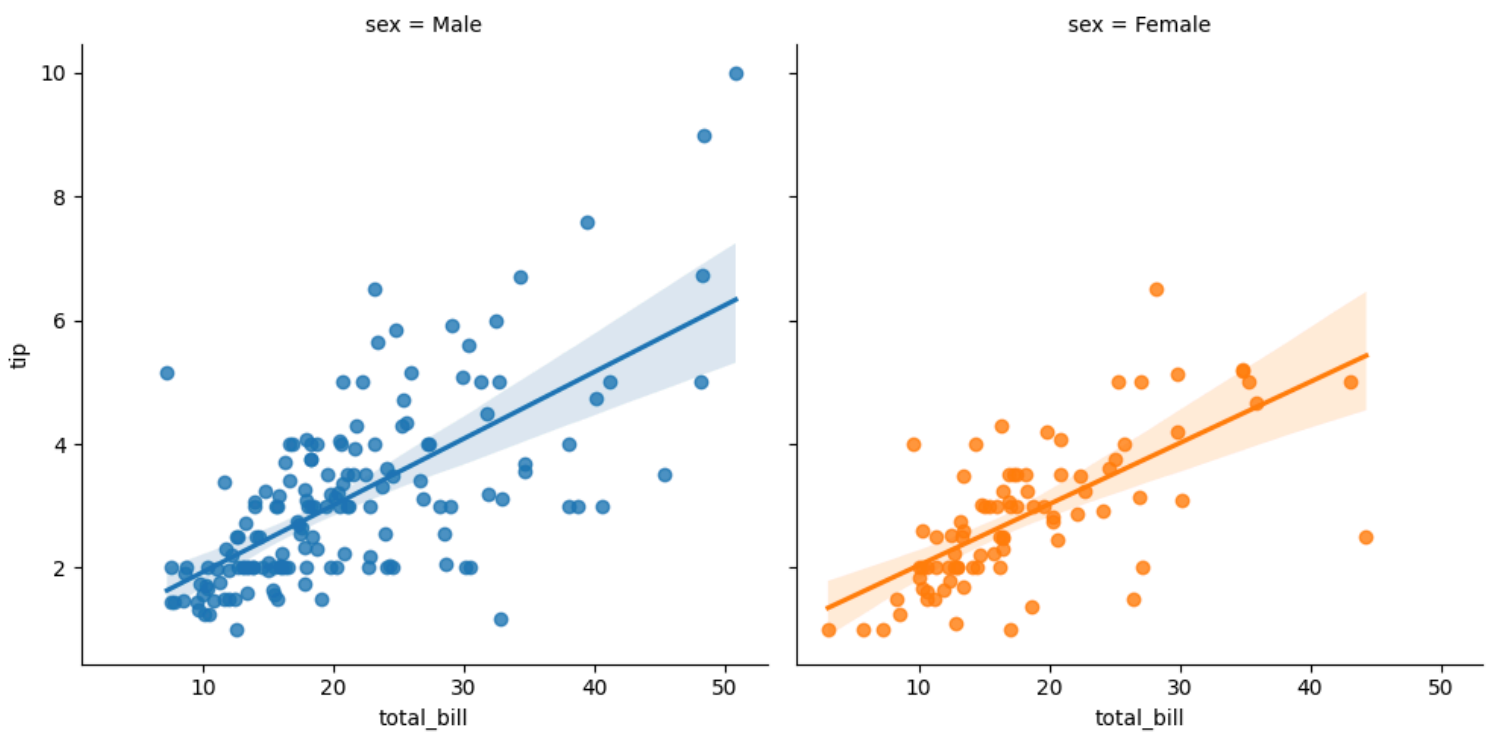
```
sns.lmplot(x='total_bill', y='tip', data=tips, col='smoker', hue='smoker')
plt.show()
```

[]:



[]:

```
sns.lmplot(x='total_bill', y='tip', data=df, fit_reg = True, col='sex', hue='sex')
plt.show()
```



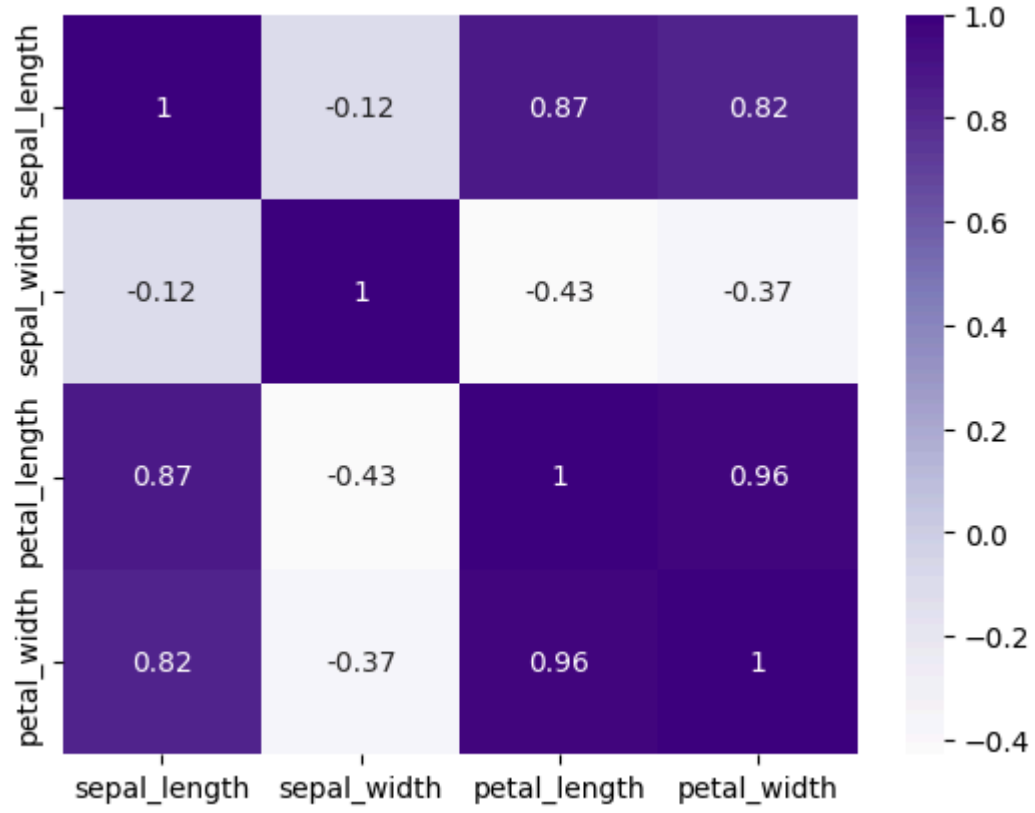
Heatmap

```
corr = iris.corr()
```

```
corr
```

	sepal_length	sepal_width	petal_length	petal_width
sepal_length	1.000000	-0.117570	0.871754	0.817941
sepal_width	-0.117570	1.000000	-0.428440	-0.366126
petal_length	0.871754	-0.428440	1.000000	0.962865
petal_width	0.817941	-0.366126	0.962865	1.000000

```
sns.heatmap(iris.corr(), cmap='Purples', annot =True)
```



Possible values are: Accent, Accent_r, Blues, Blues_r, BrBG, BrBG_r, BuGn, BuGn_r, BuPu, BuPu_r, CMRmap, CMRmap_r, Dark2, Dark2_r, GnBu, GnBu_r, Greens, Greens_r, Greys, Greys_r, OrRd, OrRd_r, Oranges, Oranges_r, PRGn, PRGn_r, Paired, Paired_r, Pastel1, Pastel1_r, Pastel2, Pastel2_r, PiYG, PiYG_r, PuBu, PuBuGn, PuBuGn_r, PuBu_r, PuOr, PuOr_r, PuRd, PuRd_r, Purples, Purples_r, RdBu, RdBu_r, RdGy, RdGy_r, RdPu, RdPu_r, RdYlBu, RdYlBu_r, RdYlGn, RdYlGn_r, Reds, Reds_r, Set1, Set1_r, Set2, Set2_r, Set3, Set3_r, Spectral, Spectral_r, Wistia, Wistia_r, YlGn, YlGnBu, YlGnBu_r, YlGn_r, YlOrBr, YlOrBr_r, YlOrRd etc

Assignment Hands-On


Screenshot%202022-10-01%20150100.png

[]:
```
from google.colab import files
uploaded = files.upload()
toyota = pd.read_csv('Toyota.csv')
toyota.head()
```

[]: Choose files No file chosen — Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

[]:
```
Saving Toyota.csv to Toyota.csv
```

[]:

	Price	Age	KM	FuelType	HP	MetColor	Automatic	CC	Doors	Weight
0	13500	23.0	46986.0	Diesel	90	1.0	0	2000	3	1165
1	13750	23.0	72937.0	Diesel	90	1.0	0	2000	3	1165
2	13950	24.0	41711.0	Diesel	90	NaN	0	2000	3	1165
3	14950	26.0	48000.0	Diesel	90	0.0	0	2000	3	1165
4	13750	30.0	38500.0	Diesel	90	0.0	0	2000	3	1170

[]:
```
toyota.info()
```

[]:
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1436 entries, 0 to 1435
Data columns (total 10 columns):
 #   Column      Non-Null Count  Dtype  
---  ------      --------------  -----  
 0   Price       1436 non-null   int64  
 1   Age         1336 non-null   float64
 2   KM          1421 non-null   float64
 3   FuelType    1336 non-null   object 
 4   HP          1436 non-null   object 
 5   MetColor    1286 non-null   float64
 6   Automatic   1436 non-null   int64  
 7   CC          1436 non-null   int64  
 8   Doors       1436 non-null   int64  
 9   Weight      1436 non-null   int64  
dtypes: float64(3), int64(5), object(2)
memory usage: 112.3+ KB
```

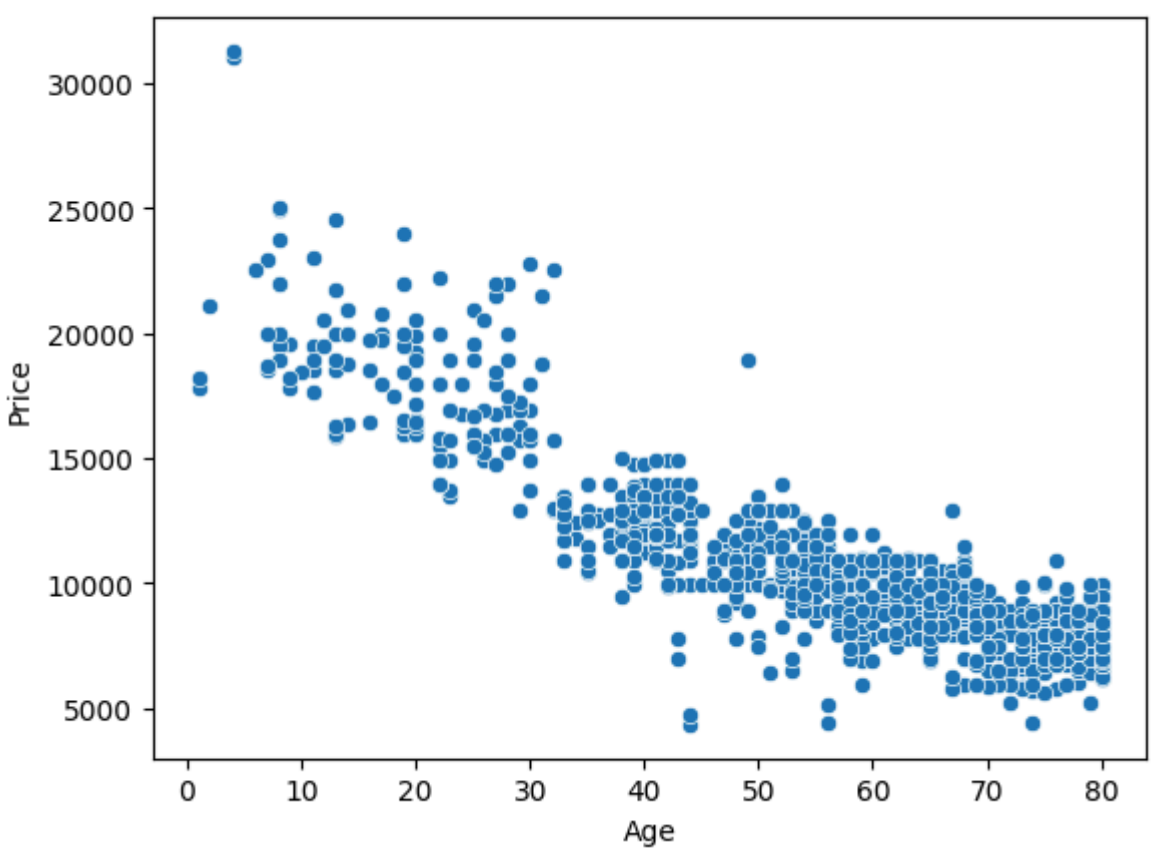
[]:
```
toyota.isnull().sum()
```

[]:
```
Price          0
Age          100
KM            15
FuelType     100
HP             0
MetColor     150
Automatic      0
CC             0
Doors          0
Weight         0
dtype: int64
```

[]:
```
toyota.dropna(inplace=True)
```

[]:
```
sns.scatterplot(x='Age', y='Price', data=toyota)
plt.show()
```

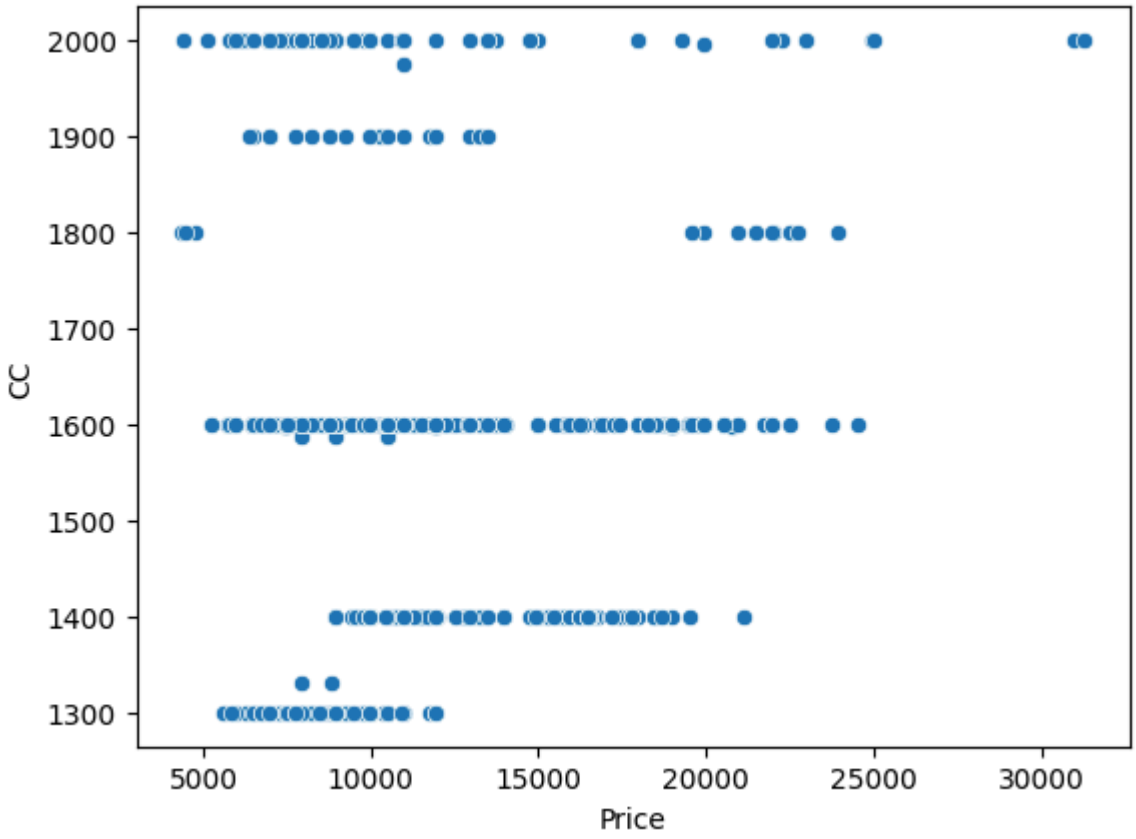
[ ]:



[ ]:

```
sns.scatterplot(x='Price', y='CC', data=toyota)
plt.show()
```

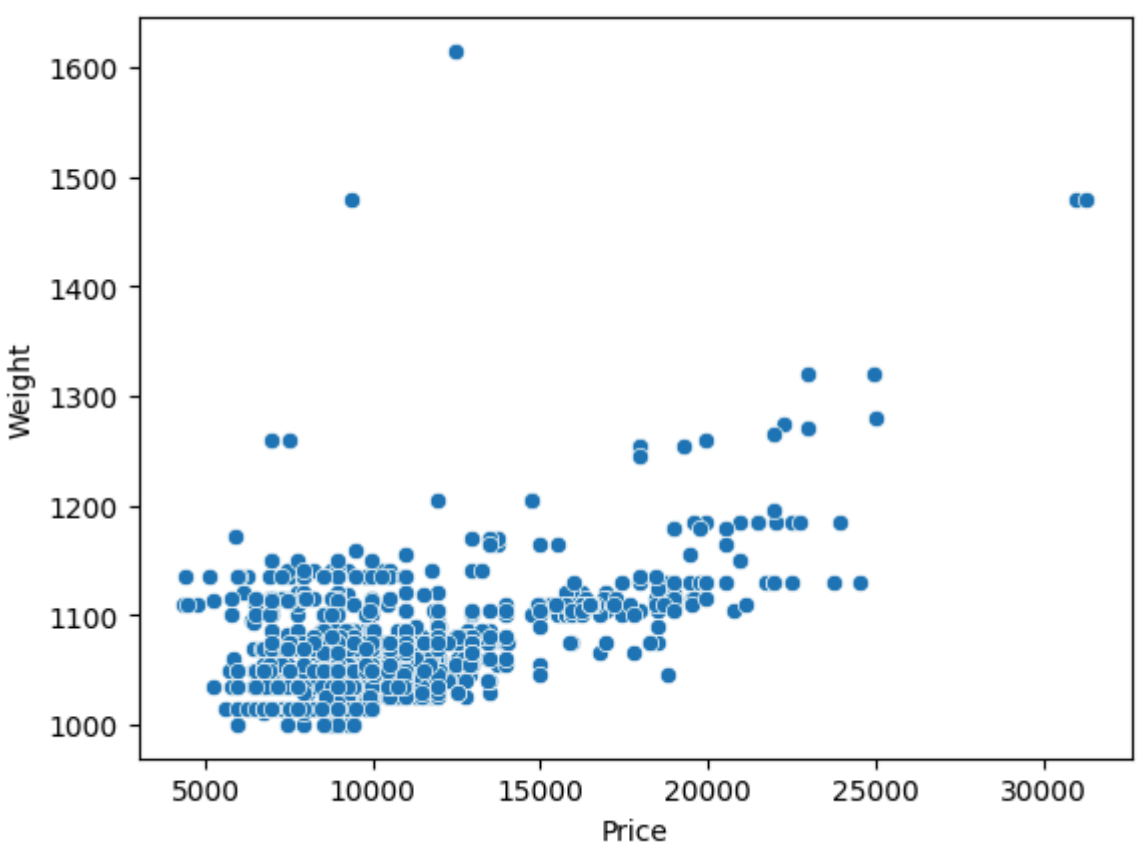
[ ]:



[ ]:

```
sns.scatterplot(x='Price', y='Weight', data=toyota)
plt.show()
```

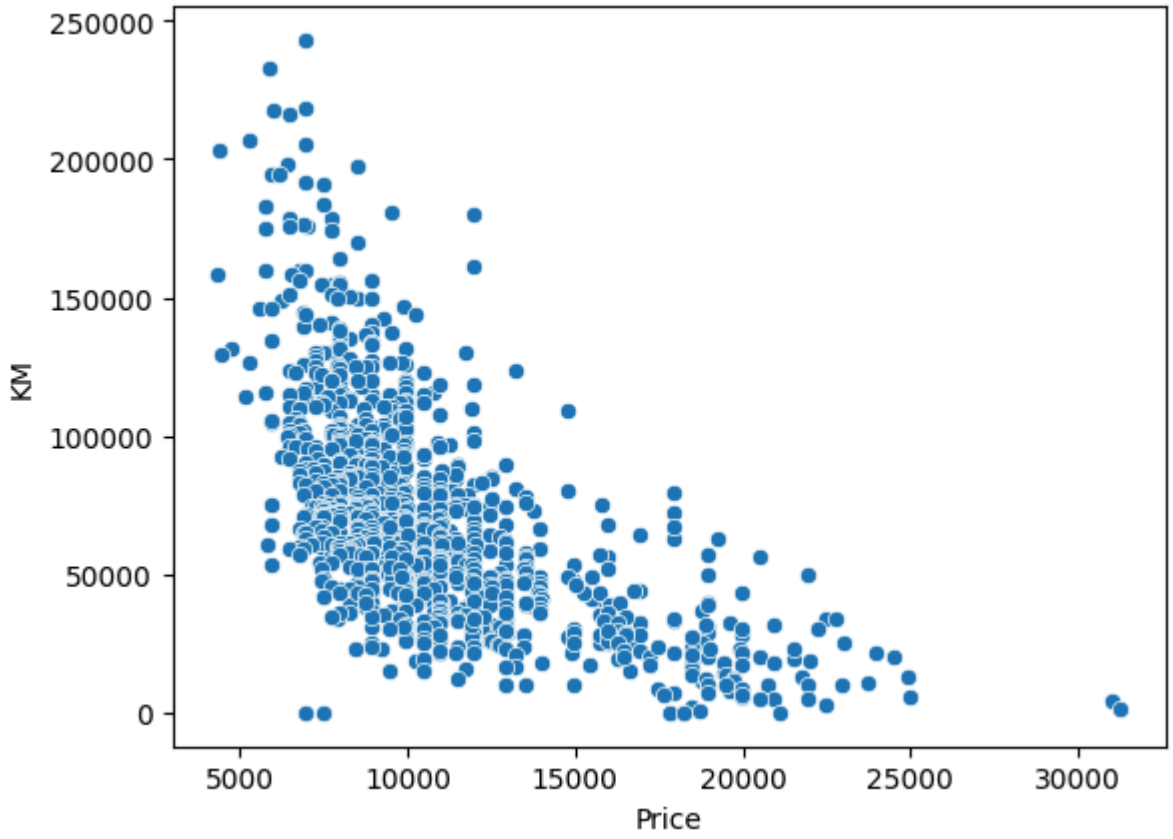
[ ]:



[ ]:

```
sns.scatterplot(x='Price', y='KM', data=toyota)
plt.show()
```

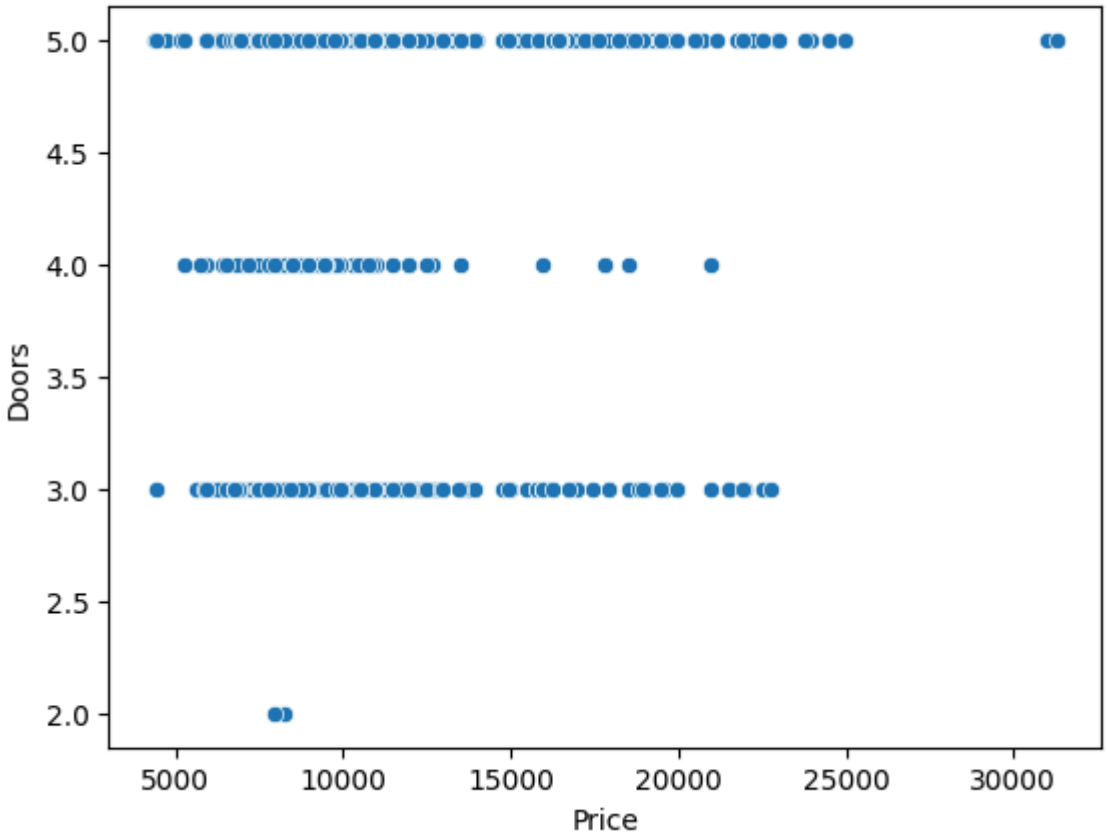
[ ]:



[ ]:

```
sns.scatterplot(x='Price', y='Doors', data=toyota)
plt.show()
```

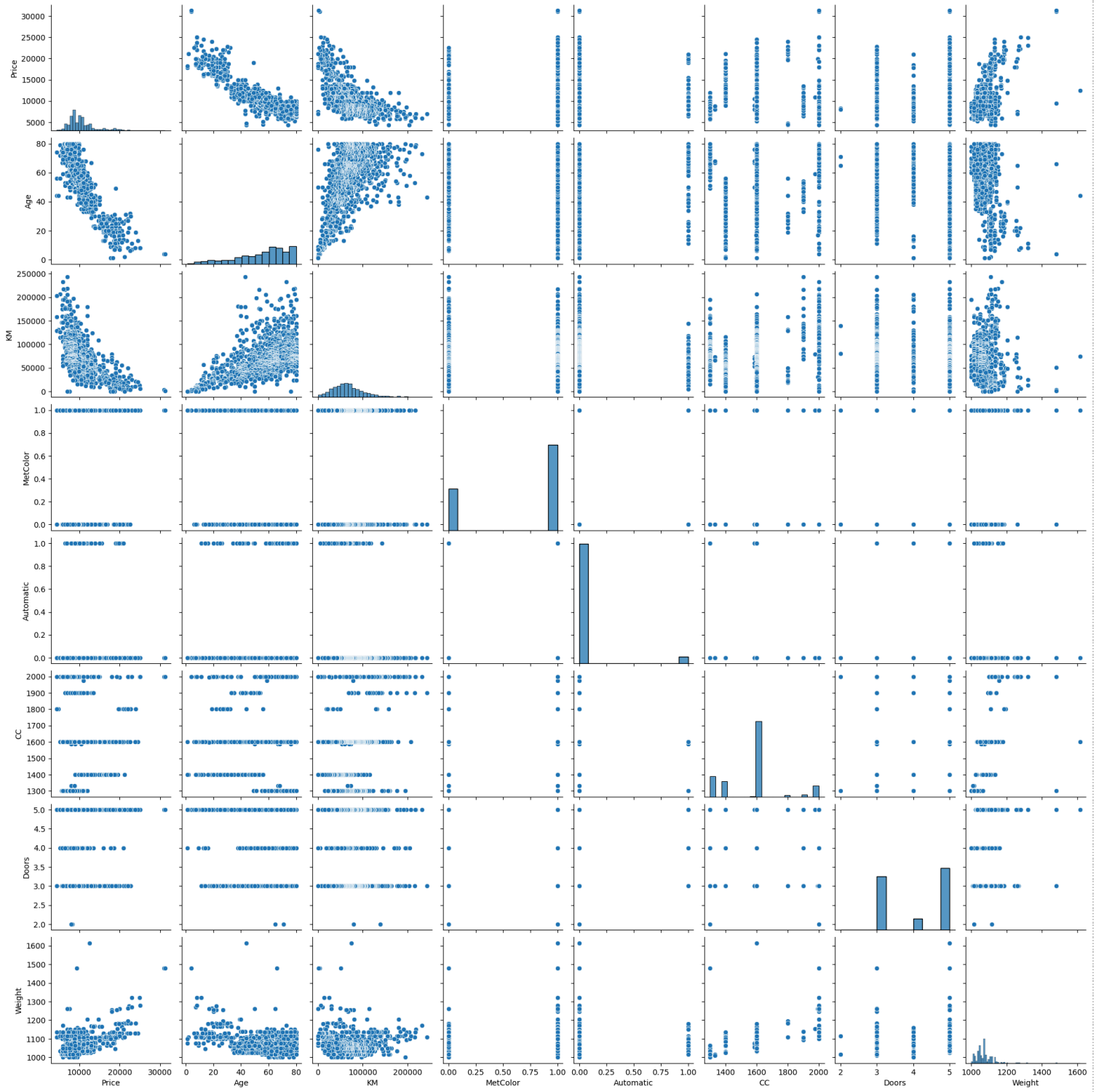
[ ]:



[ ]:
```
sns.pairplot(toyota)
```

[ ]:

[]:



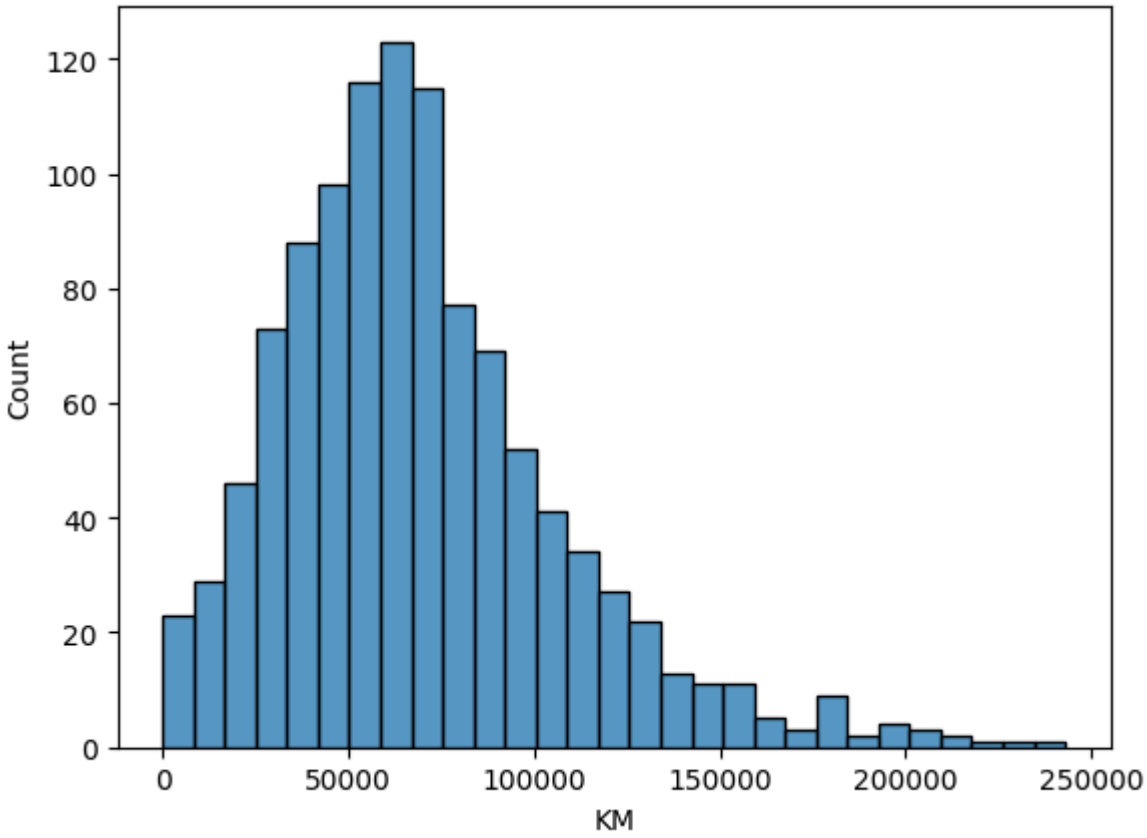
[]:

[]:

```
sns.histplot(x='KM', data=toyota)
```

[]:

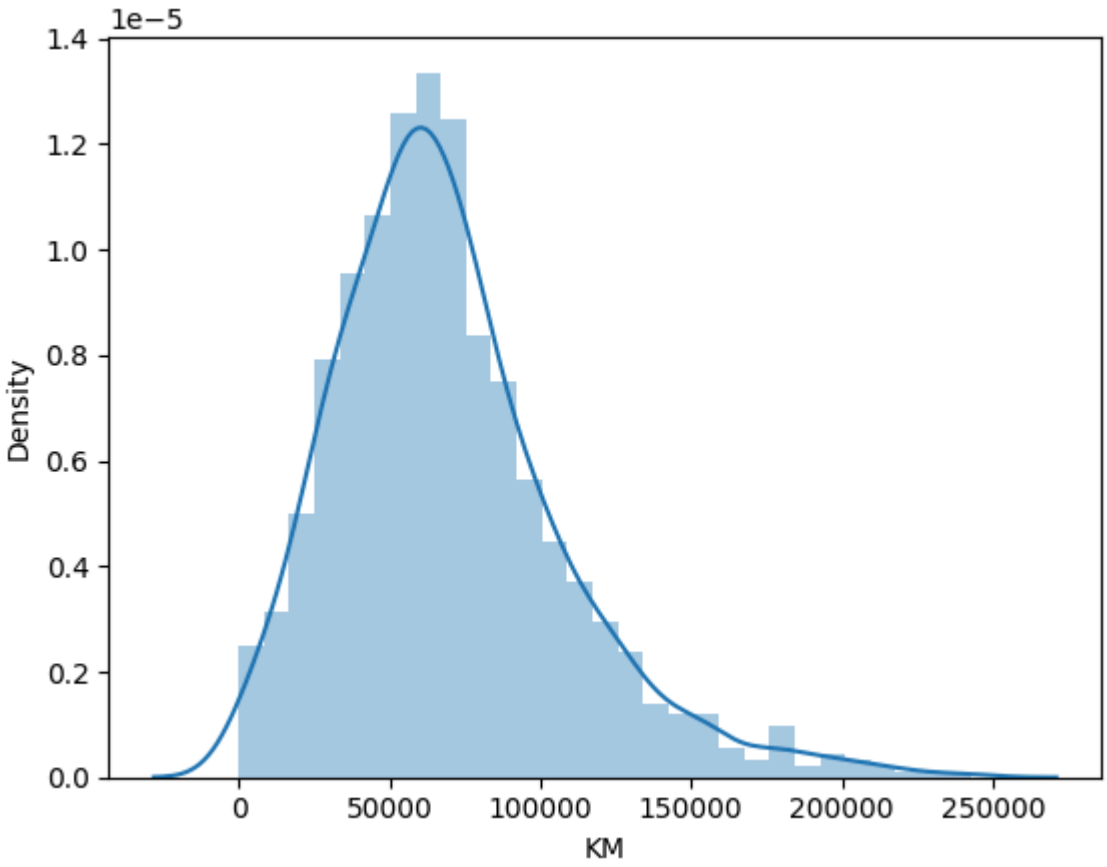
[ ]:



[ ]:

```
# 2. Histogram without kernel density estimate
sns.distplot(toyota['KM'], hist=True, kde=True)
plt.show()
```

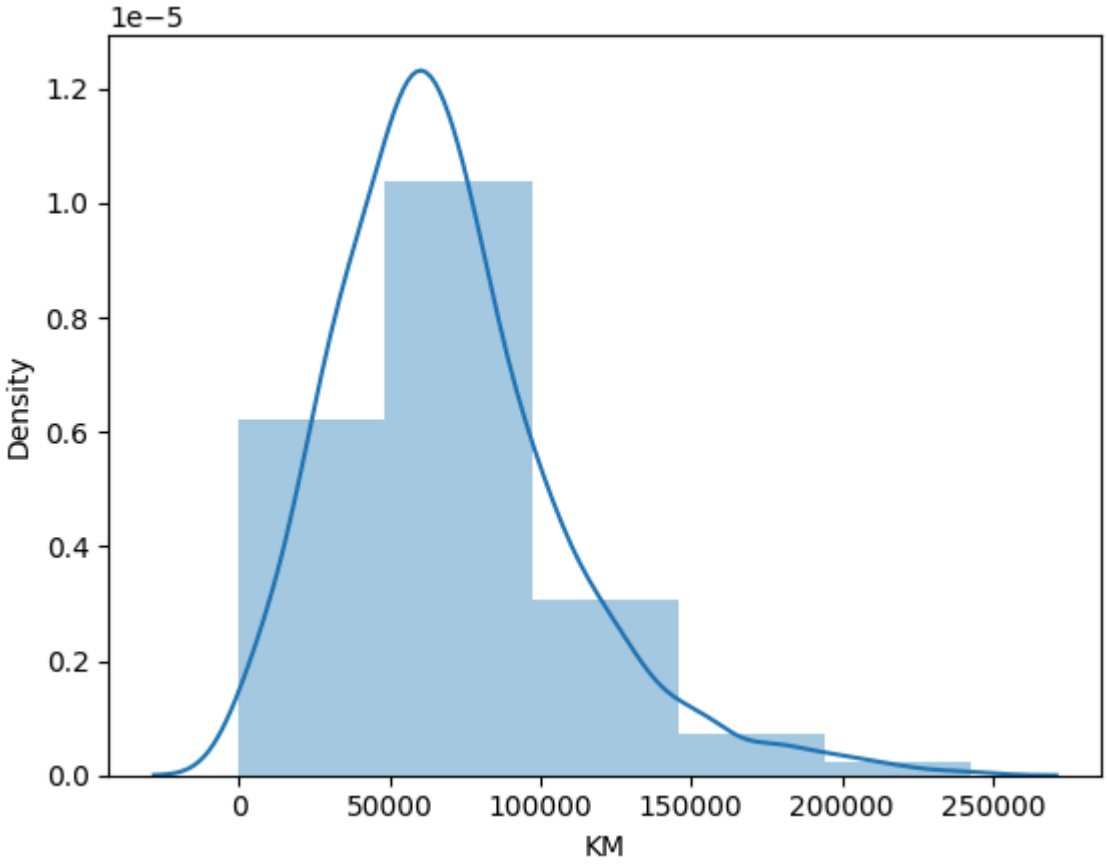
[ ]:



[ ]:

```
# 3. Histogram with fixed no. of bins
sns.distplot(toyota['KM'], bins=5 )
plt.show()
```

[ ]:



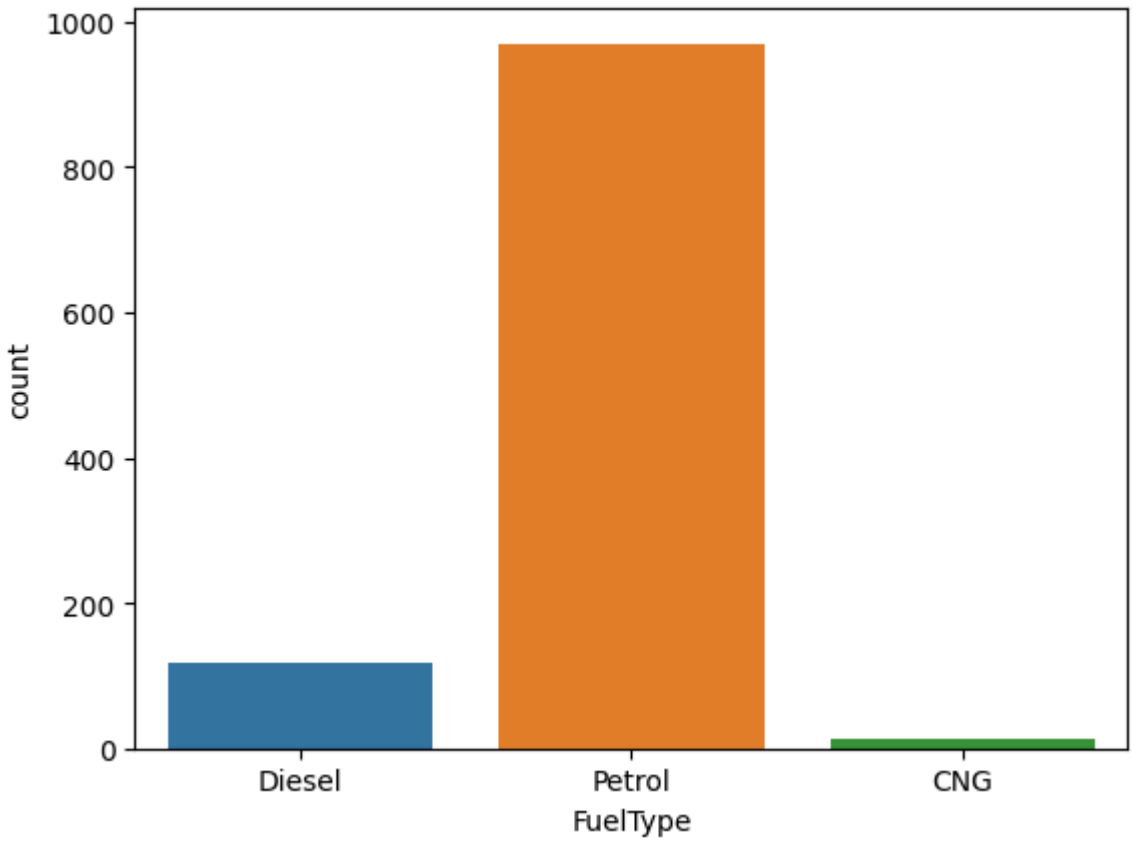
[ ]:

[ ]:

```
sns.countplot(x = 'FuelType', data=toyota)
```

[ ]:

[ ]:

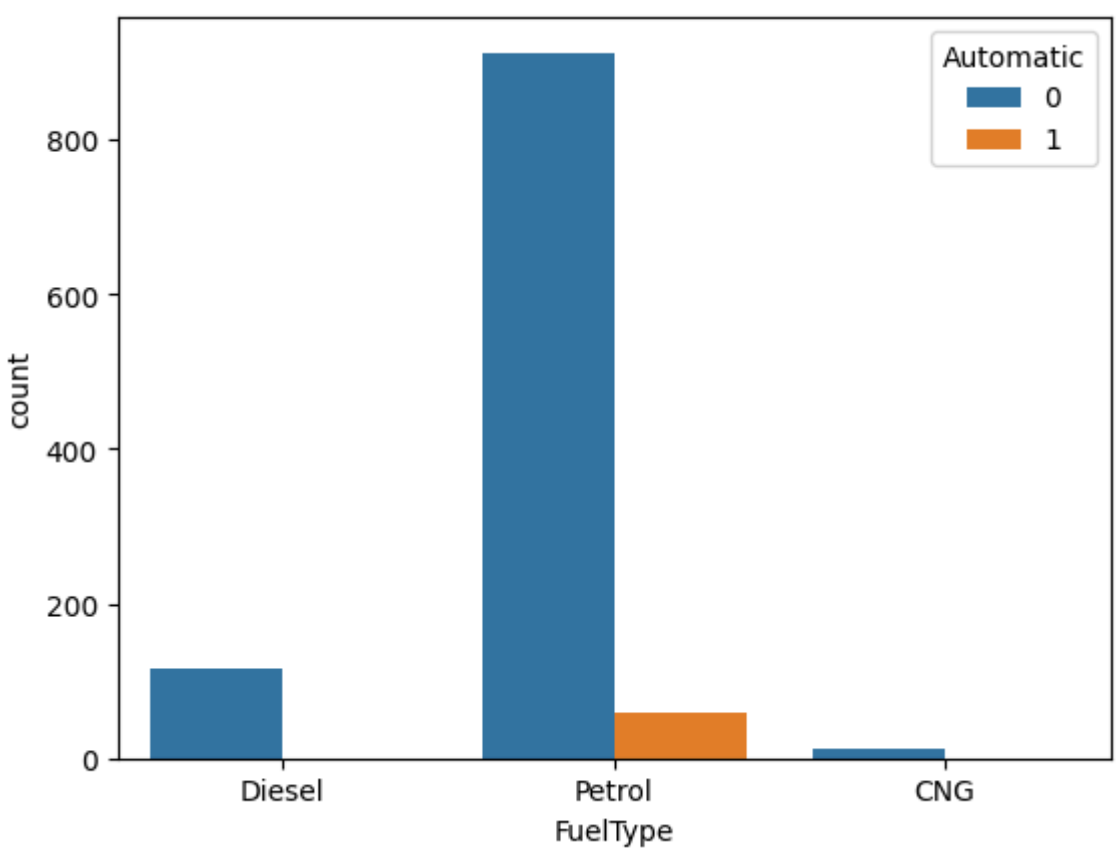


[ ]:

```
sns.countplot(x = 'FuelType', data=toyota, hue='Automatic')
```

[ ]:

[ ]:

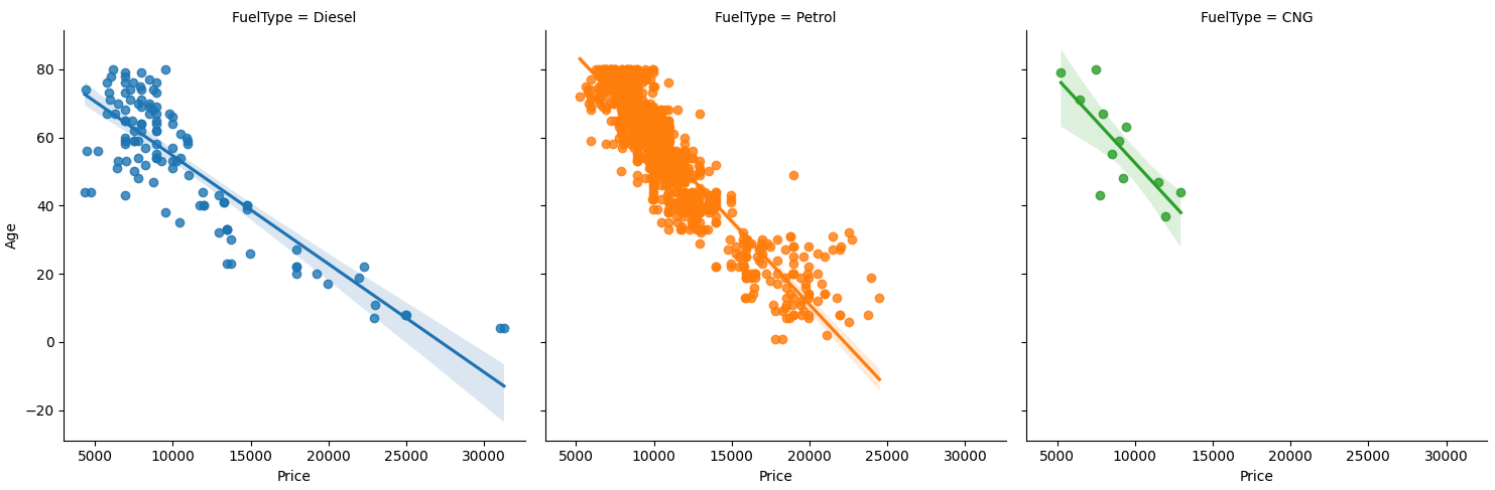


[ ]:

[ ]:

```
sns.lmplot(x='Price', y='Age', data=toyota,col='FuelType', hue='FuelType')
plt.show()
```

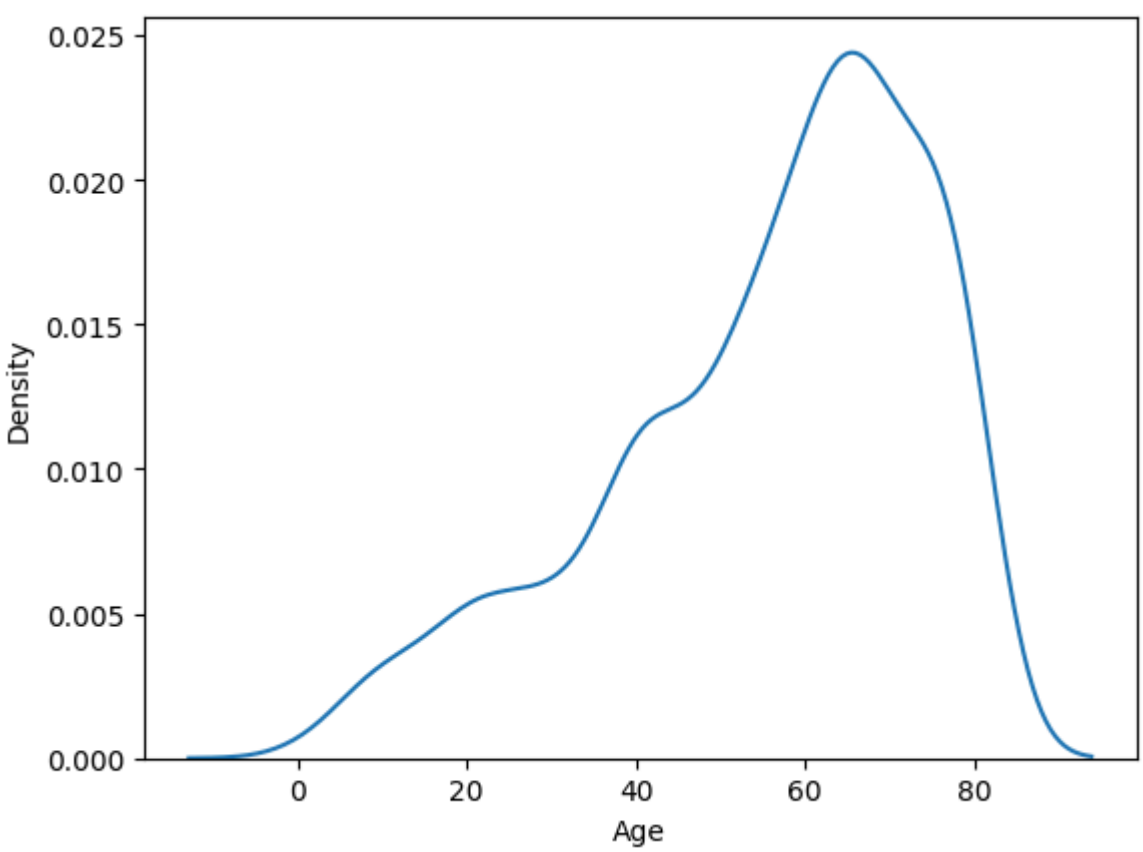
[ ]:



[ ]:

```
sns.kdeplot(x='Age', data=toyota)
plt.show()
```

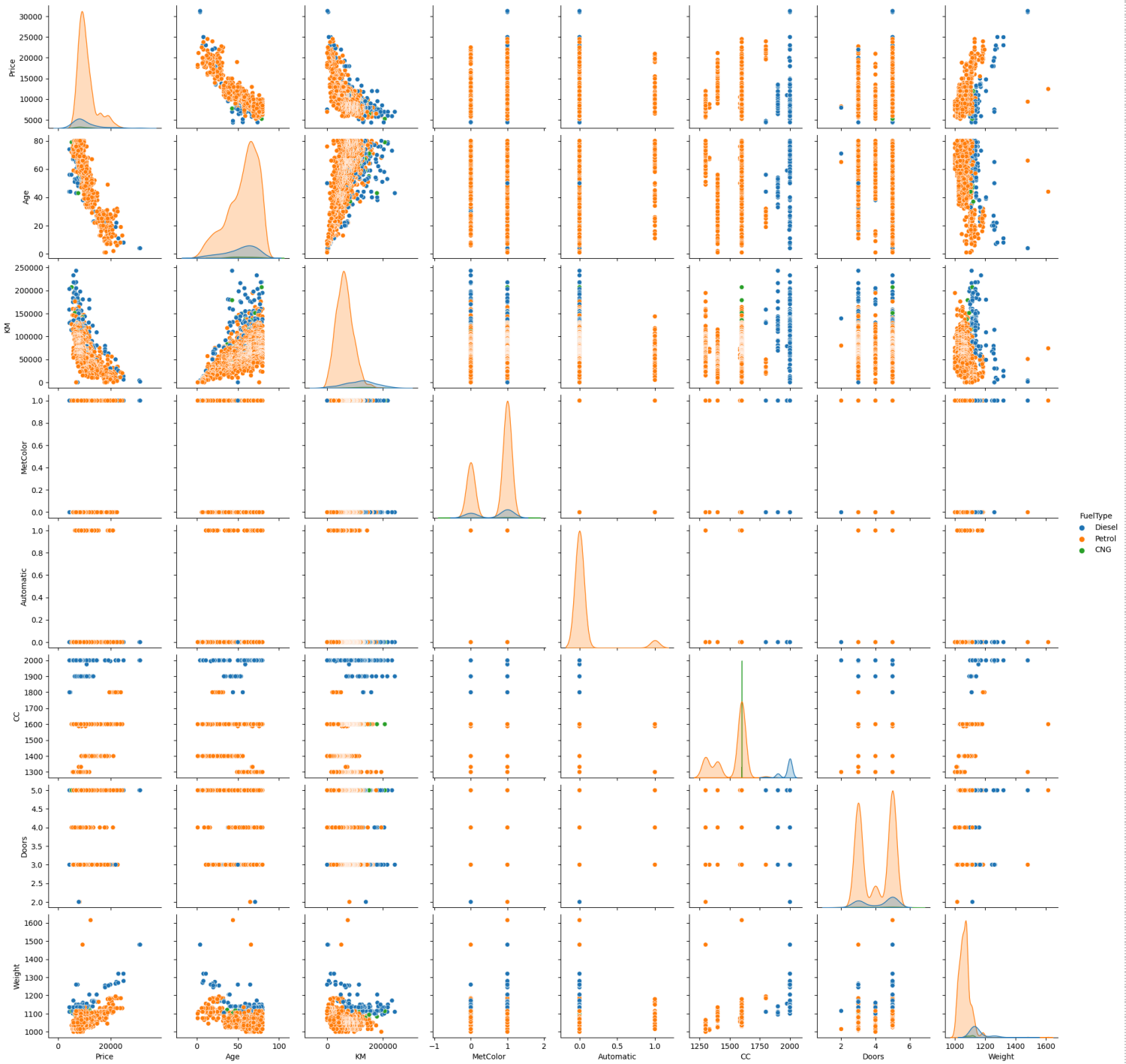
[]:



[]:
```
sns.pairplot(data=toyota, kind = "scatter",hue = "FuelType")
```

[]:

[]:


Price
Age
KM
MetColor
Automatic
CC
Doors
Weight
FuelType
Diesel
Petrol
CNG

[]: